

ZCAPNet - Sistema de apoio à Proteção Civil

Gonçalo Dimas

Luis Alves

Orientadores: André Lourenço
Artur Ferreira

Relatório da versão final do projeto realizado no âmbito de Projeto e Seminário
Licenciatura em Engenharia Informática e de Computadores
no semestre de verão 2024/2025

julho de 2025

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

ZCAPNet

48263 Gonalo Alexandre Rocha Dimas

46974 Luis Ant3nio Carvalho Alves

Orientadores: Artur Jorge Ferreira

Andr3 Ribeiro Loureno

Relat3rio da vers3o final do projeto realizado no 3mbito de Projeto e Semin3rio
Licenciatura em Engenharia Inform3tica e de Computadores
no semestre de ver3o 2024/2025

julho de 2025

Resumo

O alojamento temporário de emergência é uma valência do apoio psicossocial em emergência, essencial para proporcionar ao cidadão individual e às famílias um local seguro para permanecerem, antes, durante e após um acidente grave ou catástrofe, onde são igualmente asseguradas as suas necessidades básicas, bem como apoio psicossocial de emergência.

Assim, o alojamento de emergência providenciado em caso de acidente grave, catástrofe ou outro tipo de ocorrência de carácter excecional é uma resposta temporária à existência de cidadãos que, no âmbito de uma evacuação, são deslocados das suas residências por não estarem numa zona segura.

Esta resposta mantém-se enquanto não é restabelecida a normalidade ou os cidadãos necessitam de apoio psicossocial em emergência.

No âmbito do Sistema Integrado de Operações de Proteção e Socorro (SIOPS), do Sistema de Gestão de Operações (SGO) e dos Planos de Emergência Nacionais, Regionais, Distritais, Municipais e Especiais, a designação utilizada para o alojamento de emergência de pessoas deslocadas na sequência de acidente grave ou catástrofe é Zona de Concentração e Apoio à População (ZCAP).

Embora existam vários tipos de alojamento de emergência, um dos objetivos principais de uma ZCAP é o de constituir um local seguro para os indivíduos e famílias, afetados por uma emergência ou acidente grave, poderem pernoitar ou descansar e oferecer, entre outras, cuidados básicos de saúde, alimentação, bebidas, agasalhos, instalações sanitárias, apoio psicossocial e informações sobre o desenvolvimento das operações de socorro.

O objetivo deste projeto é criar um sistema computacional centralizado para assistir na gestão das ZCAP garantindo a existência de centralização de informação com uma estrutura de dados uniforme.

A arquitetura desenhada baseia-se na implementação de uma aplicação desktop autónoma sob o princípio de funcionamento *offline-first* com uma base de dados local e sincronização posterior com a base de dados remota a partir de uma Web API desenvolvida para esse efeito.

O sistema desenvolvido tem capacidade de resiliência na eventualidade de falha das infraestruturas de comunicação e permite ser evoluído para casos de uso futuro.

Palavras-chave: Alojamento Temporário de emergência, Apoio Psicossocial em emergência,

Proteção Civil, *offline-first*, ZCAP.

Abstract

Emergency temporary shelter is a component of psychosocial emergency support, essential for providing individuals and families with a safe place to stay before, during, and after a serious accident or disaster. It ensures that their basic needs are met, while also delivering psychosocial emergency support.

Thus, emergency shelter provided in the event of a serious accident, disaster, or other exceptional occurrence is a temporary response to support citizens who, following an evacuation, have been displaced from their homes due to unsafe conditions, until normality is restored or psychosocial support is no longer required.

Within the framework of the Integrated System for Protection and Relief Operations (SI-OPS), the Operations Management System (SGO), and National, Regional, District, Municipal, and Special Emergency Plans, the designation used for emergency shelter for displaced persons is Population Support and Assembly Zone (ZCAP).

Although there are various types of emergency accommodation, one of the main objectives of a ZCAP is to provide a safe location where individuals and families affected by an emergency or serious accident can sleep or rest. ZCAPs also offer, among other services, basic healthcare, food, beverages, warm clothing, sanitary facilities, psychosocial support, and updates of rescue operations.

The objective of this project is to develop a centralized computational system to support the management of ZCAPs, ensuring uniform data structuring and centralized information.

The proposed architecture is based on the implementation of a standalone desktop application under an *offline-first* paradigm, using a local database with subsequent synchronization to a remote database through a custom-built Web API.

The developed system is resilient in the event of communication infrastructure failures and is designed to evolve for future use cases.

Keywords: Civil Protection, Emergency Psychosocial Support, Emergency Temporary Shelter, *Offline-first*, ZCAP.

Lista de Acrónimos

ANEPC Autoridade Nacional de Emergência e Proteção Civil

API Application Programming Interface

CRUD Create Read Update Delete

DTO Data Transfer Object

HTTP Hypertext Transfer Protocol

JPA Jakarta Persistence API

JSON JavaScript Object Notation

JWT JSON Web Token

NISS Número de identificação da Segurança Social

ORM Object-relational mapping

REST Representational State Transfer

SGO Sistema de Gestão de Operações

SHA Secure Hash Algorithm

SIOPS Sistema Integrado de Operações de Proteção e Socorro

SQL Structured Query Language

UI User Interface

ZCAP Zona de Concentração e Apoio à População

Índice

Resumo	i
Abstract	iii
Acrónimos	v
Lista de Figuras	xii
1 Introdução	1
1.1 Objetivo	2
1.2 Organização do documento	2
2 Solução a desenvolver	5
2.1 Sistemas semelhantes	5
2.2 Arquitetura da solução	6
2.2.1 Desktop App	6
2.2.2 Web API	6
2.2.3 Base de Dados	6
2.2.4 Base de Dados Local	6
2.2.5 Resumo da Arquitetura	7
3 Modelo de Dados	9
3.1 Abordagem	9
3.2 Estrutura do modelo	9
3.2.1 Estrutura Hierárquica	10
3.2.2 Pessoas	12
3.2.3 Utilizadores	15
3.2.4 ZCAPs	17
3.2.5 Incidentes	19
3.3 Extensibilidade e Escalabilidade	21

4	Camada de apresentação - <i>Frontend</i>	23
4.1	Introdução	23
4.2	Estrutura do Projeto	24
4.2.1	Dependências Principais	25
4.2.2	Desenvolvimento	25
4.2.3	Modo <i>Offline-First</i> e persistência de Dados	26
4.3	A Aplicação	26
4.3.1	Acesso à aplicação	26
4.3.2	Multi-língua	27
4.3.3	Árvore hierárquica	27
4.3.4	ZCAPS	28
4.3.5	Incidentes	28
4.3.6	Pessoas	28
4.3.7	Gestão de Utilizadores	30
4.3.8	Sincronização	32
5	Camada de Processamento - <i>Backend</i>	35
5.1	Estrutura e Organização	35
5.1.1	Entidades	36
5.1.2	DTOs (<i>Data Transfer Objects</i>) e Repositórios	37
5.1.3	Serviços	39
5.1.4	Controladores	39
5.2	<i>Endpoints</i>	39
5.3	Estrutura de Respostas	41
5.3.1	Respostas de Sucesso	41
5.3.2	Respostas de erro	41
5.4	Documentação	42
6	Avaliação experimental	43
6.1	Testes Unitários	43
6.1.1	Backend	43
6.1.2	Frontend	43
6.2	Testes de Aceitação com o Cliente	43
7	Conclusão	45
	Referências	50

A	Lista de <i>Endpoints</i> da Web API	51
A.1	Incidentes	51
A.2	Pessoas	52
A.3	ZCAPs	55
A.4	Árvore hierárquica	57
A.5	Utilizadores	59
A.6	Outros endpoints	61

Lista de Figuras

1.1	Integração do sistema ZCAPNet no processo de evacuação e acolhimento em ZCAPs	2
2.1	Arquitetura da solução proposta.	7
3.1	Representação do modelo do nível das divisões administrativas.	11
3.2	Representação do modelo dos elementos da estrutura.	11
3.3	Representação do modelo dos tipos de detalhes.	11
3.4	Representação do modelo dos detalhes.	12
3.5	Representação do modelo dos detalhes.	12
3.6	Representação do modelo dos destinos de partida de uma ZCAP.	13
3.7	Representação do modelo das pessoas.	13
3.8	Representação do modelo das necessidades especiais.	14
3.9	Representação do modelo da relação da necessidade especial com uma pessoa.	14
3.10	Representação do modelo do tipo de relações de parentesco.	14
3.11	Representação do modelo de graus de parentesco.	15
3.12	Representação do modelo das permissões por região aceites pelo sistema.	15
3.13	Representação do modelo da associação da região territorial com os perfis de permissões de dados.	15
3.14	Representação do modelo dos perfis de utilizador.	16
3.15	Representação do modelo das chaves de acesso.	16
3.16	Representação do modelo da associação chaves de acesso-perfil de utilizador.	16
3.17	Representação do modelo dos utilizadores.	17
3.18	Representação do modelo tipo de entidades.	17
3.19	Representação do modelo das entidades.	17
3.20	Representação do modelo dos tipos de edifícios.	18
3.21	Representação do modelo das ZCAPs.	18
3.22	Representação do modelo das categorias de detalhe das ZCAPs.	19
3.23	Representação do modelo dos tipos de detalhe das ZCAPs.	19
3.24	Representação do modelo dos detalhes das ZCAPs.	19
3.25	Representação do modelo dos tipos de incidentes.	19

3.26	Representação do modelo dos incidentes.	20
3.27	Representação do modelo da associação incidente-ZCAP.	20
3.28	Representação do modelo da associação incidente-ZCAP-pessoas.	20
4.1	Fluxograma de acesso à aplicação	27
4.2	Exemplos de uso da <code>animated_tree_view</code>	28
4.3	ZCAP - Modelo conceptual de uma ZCAP: associação à árvore e detalhe por categorias	29
4.4	ZCAPs no contexto de um Incidente	30
4.5	Pessoas - Esquema de utilização	31
4.6	Perfil - Exemplo de permissões	32
4.7	Fluxo de Sincronização de dados	33
5.1	Exemplo da entidade relacional dos utilizadores.	37
5.2	Exemplos dos DTOs dos utilizadores	38
5.3	Exemplo do Repositório dos utilizadores.	38
5.4	Organização dos <i>Endpoints</i>	40
5.5	Exemplo do controlador dos utilizadores.	40

Lista de Tabelas

A.1	Tabela de <i>endpoints</i> relativos aos tipos de incidentes.	51
A.2	Tabela de <i>endpoints</i> relativos aos incidentes.	51
A.3	Tabela de <i>endpoints</i> relativos às ZCAPs associadas a um incidente.	52
A.4	Tabela de <i>endpoints</i> relativos às pessoas associadas a um incidente-ZCAP.	52
A.5	Tabela de <i>endpoints</i> relativos destinos de partida de uma ZCAP.	53
A.6	Tabela de <i>endpoints</i> relativos às pessoas.	53
A.7	Tabela de <i>endpoints</i> relativos às necessidades especiais.	53
A.8	Tabela de <i>endpoints</i> relativos à lista de necessidades especiais de cada pessoa.	54
A.9	Tabela de <i>endpoints</i> relativos aos suportes necessários.	54
A.10	Tabela de <i>endpoints</i> relativos à lista de suporte necessário de cada pessoa.	54
A.11	Tabela de <i>endpoints</i> relativos ao tipos de graus de parentesco.	55
A.12	Tabela de <i>endpoints</i> relativos aos graus de parentesco entre pessoas.	55
A.13	Tabela de <i>endpoints</i> relativos aos tipos de edifício de ZCAPs.	55
A.14	Tabela de <i>endpoints</i> relativos aos tipos de entidade que gerem uma ZCAP.	56
A.15	Tabela de <i>endpoints</i> relativos às entidades que gerem uma ZCAP.	56
A.16	Tabela de <i>endpoints</i> relativos às ZCAPs.	56
A.17	Tabela de <i>endpoints</i> relativos às categorias dos detalhes das ZCAPs.	56
A.18	Tabela de <i>endpoints</i> relativos aos tipos de detalhe das ZCAPs.	57
A.19	Tabela de <i>endpoints</i> relativos aos detalhes das ZCAPs.	57
A.20	Tabela de <i>endpoints</i> relativos aos níveis das divisões administrativas.	57
A.21	Tabela de <i>endpoints</i> relativos aos elementos da árvore hierárquica.	58
A.22	Tabela de <i>endpoints</i> relativos aos tipos de detalhe dos elementos da árvore hierárquica.	58
A.23	Tabela de <i>endpoints</i> relativos aos detalhes dos elementos da árvore hierárquica.	58
A.24	Tabela de <i>endpoints</i> relativos à associação do nível como o tipo de detalhes dos elementos da árvore hierárquica.	59
A.25	Tabela de <i>endpoints</i> relativos aos perfis de dados dos utilizadores.	59
A.26	Tabela de <i>endpoints</i> relativos aos detalhes dos perfis de dados dos utilizadores.	60
A.27	Tabela de <i>endpoints</i> relativos aos perfis dos utilizadores.	60
A.28	Tabela de <i>endpoints</i> relativos às chaves de acesso aos menus.	60

A.29 Tabela de <i>endpoints</i> relativos às lista de permissões de cada perfil de utilizador.	60
A.30 Tabela de <i>endpoints</i> relativos aos utilizadores.	60
A.31 Tabela de <i>endpoints</i> relativos aos utilizadores para uso da aplicações <i>offline-first</i> .	61
A.32 Tabela de <i>endpoints</i> relativos aos dados do utilizador.	61

Capítulo 1

Introdução

O alojamento temporário de emergência é uma valência do apoio psicossocial em emergência, essencial para proporcionar ao cidadão individual e às famílias um local seguro para permanecerem, antes, durante e após um acidente grave ou catástrofe, onde são igualmente asseguradas as suas necessidades básicas, bem como apoio psicossocial de emergência.

Assim, o alojamento de emergência providenciado em caso de acidente grave, catástrofe ou outro tipo de ocorrência de carácter excecional é uma resposta temporária à existência de cidadãos que, no âmbito de uma evacuação, são deslocados das suas residências por não estarem numa zona segura, enquanto não é restabelecida a normalidade ou necessitam de apoio psicossocial em emergência.

No âmbito do Sistema Integrado de Operações de Proteção e Socorro (SIOPS) [1], do Sistema de Gestão de Operações (SGO) [2] e dos Planos de Emergência Nacionais, Regionais, Distritais, Municipais e Especiais, a designação utilizada para o alojamento de emergência de pessoas deslocadas na sequência de acidente grave ou catástrofe é **Zona de Concentração e Apoio à População (ZCAP)** [3].

Existem vários tipos de alojamento de emergência:

- Alojamentos dispersos
 - Alojamento arrendado
 - Famílias de acolhimento - casa de amigos/familiares/famílias voluntárias
 - Habitação própria - pessoas permanecem na sua casa apesar do dano ou risco
 - Alojamento de lazer - permanência em unidades hoteleiras, casas de férias, parques de campismo
- ZCAP
 - Equipamentos Sociais - Pavilhões desportivos, escolas, centros comunitários
 - Campo de tendas

- Campo de pré-fabricados - casas pré-fabricadas em terrenos com serviços comunitários limitados

Um dos objetivos principais de uma ZCAP é o de constituir um local seguro para os indivíduos e famílias, afetados por uma emergência ou acidente grave, poderem pernoitar ou descansar.

Estes espaços oferecem também, entre outras, cuidados básicos de saúde, alimentação, bebidas, agasalhos, instalações sanitárias, apoio psicossocial e informações sobre o desenvolvimento das operações de socorro.

1.1 Objetivo

O objetivo deste projeto é criar um sistema computacional centralizado para assistir na gestão das ZCAP garantindo a existência de centralização de informação com uma estrutura de dados uniforme (Fig. 1.1).

A arquitetura desenhada baseia-se na implementação de uma aplicação *desktop* autónoma sob o princípio de funcionamento *offline-first* com uma base de dados local e sincronização posterior com a base de dados remota a partir de uma Web API desenvolvida neste trabalho.

O sistema desenvolvido tem capacidade de resiliência na eventualidade de falha das infraestruturas de comunicação e permite ser evoluído para casos de uso futuro.



Figura 1.1: Integração do sistema ZCAPNet no processo de evacuação e acolhimento em ZCAPs

1.2 Organização do documento

O presente relatório tem como objetivo descrever a forma como foi idealizada e implementada a ideia de solução para o problema apresentado.

Nos capítulos que se seguem, são apresentados os componentes desenvolvidos, bem como as principais escolhas e decisões tomadas durante a sua realização.

O segundo capítulo descreve a solução desenvolvida ao longo do semestre.

O terceiro incide sobre o modelo de dados, assim como a sua estrutura e tipos de abordagem adotados.

O quarto capítulo é dedicado à aplicação *frontend*, através da qual os utilizadores interagem com o sistema.

O quinto capítulo descreve a solução implementada na camada de processamento - *backend* responsável por armazenar, processar e disponibilizar dados de forma eficiente, assegurando a comunicação entre a base de dados e a interface com o utilizador.

O sexto capítulo é reservado à componente de Avaliação Experimental e por último, no sétimo capítulo, são discutidas as conclusões, observações, objetivos cumpridos e propostas de melhoria ou continuidade futura.

Capítulo 2

Solução a desenvolver

O documento orientador da Instalação e Gestão de Zonas de Concentração e Apoio à População (ZCAP) [3], serviu de base à análise do problema a resolver com este projeto. Este, descreve todos os procedimentos necessários para o funcionamento destas instalações, antes, durante e após um acidente grave ou catástrofe.

Da análise efetuada, considerou-se relevante a necessidade de garantir o funcionamento, mesmo na eventualidade de uma suspensão de serviço das infraestruturas de comunicação, de uma aplicação onde seja possível registar e manter o apoio prestado e os seus utilizadores.

Assim, foi decidido que a arquitetura da solução, teria necessariamente de garantir que, apesar de a informação ser armazenada numa base de dados global e centralizada, servida e exposta por uma Web API, os clientes desta teriam de localmente conter informação suficiente para serem resilientes a esta falha eventual.

A secção [2.1 Sistemas semelhantes](#) descreve a realidade existente e as lacunas detetadas que motivaram também a decisão de escolha da **Arquitetura** escolhida descrita na secção [2.2](#).

2.1 Sistemas semelhantes

A Autoridade Nacional de Emergência e Proteção Civil (ANEPC) utiliza a aplicação web SIGO - Sistema Integrado de Gestão de Operações [4]. Esta aplicação contém funcionalidades de suporte à instalação e gestão de ZCAPs; no entanto, foram identificadas lacunas neste sistema, as quais o projeto visa responder, nomeadamente:

- A aplicação é utilizada apenas por agentes da ANEPC e apenas quando é atingido uma determinada fase do Sistema de Gestão de Operações (SGO) [2] no qual, dada a complexidade e recursos envolvidos, o Comando das Operações passa necessariamente para esta entidade. Nesta fase é possível que já estejam implementadas ZCAPs, as quais são geridas por outras entidades, sem acesso ao software.
- A inexistência de dados das estruturas existentes não permite apurar estatisticamente

a capacidade de resposta em situação de crise.

- O software é acessado por browser e depende da existência de conectividade com a sua Web API.

2.2 Arquitetura da solução

A solução proposta segue uma arquitetura **modular e integrada**, composta por três camadas principais: a aplicação Desktop (*Desktop App*), a API de serviço (*Web API*) e a base de dados central (*Database*).

2.2.1 Desktop App

A aplicação será desenvolvida em **Flutter** [5], uma *framework* multi-plataforma que, embora criada para desenvolver aplicações nativas para Android e iOS a partir de um único código-base, suporta da mesma forma o desenvolvimento para Desktop quer seja para Windows, Linux ou MacOS. A escolha inicial de tecnologia foi o *React Native* por ter eficiência e desempenho semelhante ao **Flutter** e a prévia experiência com **JavaScript** facilitaria o ritmo dos trabalhos. No entanto, após algum esforço percebeu-se que o suporte para Desktop e para o armazenamento local não tem ainda bibliotecas que permitissem em tempo útil a conclusão da mesma, pelo que se mudou a abordagem para **Flutter**. Foi também avaliada a opção *Kotlin Multiplatform* tendo sido descartada pela performance relativa ao **Flutter**.

2.2.2 Web API

A camada de serviços será construída com **Spring Boot**, utilizando a linguagem **Kotlin** para o desenvolvimento do *backend*. A *Web API* será responsável por expor os serviços REST, sendo documentada e especificada através de ferramentas como **Swagger** e **OpenAPI Specification**. A segurança e a autenticação dos utilizadores serão geridas através de mecanismos de autorização e autenticação JWT.

2.2.3 Base de Dados

A camada de persistência de dados será suportada por uma instância **Microsoft SQL Server**, garantindo escalabilidade, fiabilidade e integração com as ferramentas existentes. A comunicação entre a API e a base de dados será realizada com recurso à biblioteca **JPA** com **Hibernate**, simplificando desta forma o mapeamento objeto-relacional.

2.2.4 Base de Dados Local

Um dos requisitos fundamentais da aplicação é a sua resiliência e capacidade de funcionar em modo *offline-first* [6] [7]. Para tal, será implementado um mecanismo de **persistência de dados local** com recurso à biblioteca *Isar* [8], permitindo que os dados essenciais para

o funcionamento da aplicação fiquem armazenados e disponíveis localmente. Desta forma, mesmo em cenários de falha de conectividade com a API, a aplicação poderá continuar a funcionar e registar operações, garantindo a continuidade do serviço. Quando a ligação for restabelecida, os dados serão sincronizados com a base de dados central.

2.2.5 Resumo da Arquitetura

A arquitetura da solução é ilustrada na Figura 2.1, onde se destaca a separação dos seus componentes, à esquerda o *backend* composto pela Web API e base de dados, à direita os diversos clientes, cada um com a sua aplicação *frontend*, base de dados local e serviço de sincronização.

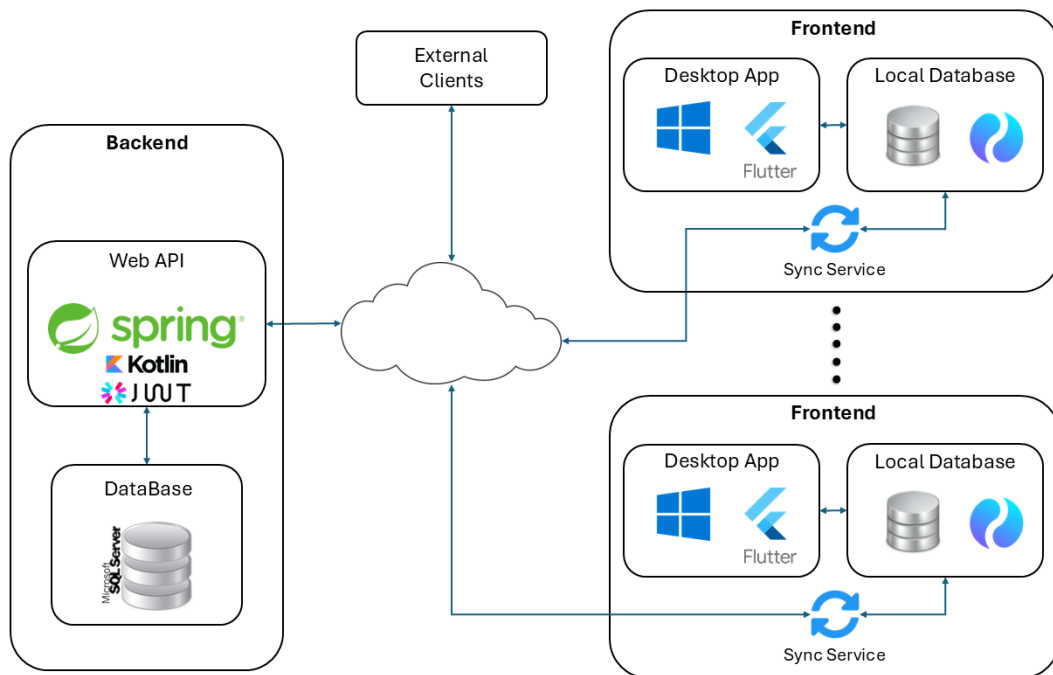


Figura 2.1: Arquitetura da solução proposta.

Capítulo 3

Modelo de Dados

O modelo de dados foi concebido para suportar a gestão e sincronização de zonas de apoio à população (ZCAP), garantindo a integridade e a acessibilidade dos dados em contextos distribuídos e *offline*.

As secções deste capítulo estão organizadas por forma a descrever na secção 3.1 a abordagem utilizada para na secção 3.2 descrever os principais grupos do modelo e as características comuns de todos para posteriormente descrever de forma individual, nas subsecções, o detalhe de cada um deles. Finalmente na secção 3.3 descreve-se o motivo da escolha do modelo e as vantagens que este apresenta por ser flexível e adaptável a novas realidades e mudanças na utilização dos dados.

3.1 Abordagem

Foi utilizada a notação entidade-relacionamento com base na metodologia de Chen [9]. O modelo foi normalizado até à 3ª forma normal para eliminar redundâncias.

A modelação da base de dados foi realizada com base no modelo entidade-relacionamento (ER), tendo como objetivo representar de forma clara e estruturada os elementos essenciais do sistema e as suas inter-relações. Esta abordagem permitiu identificar as entidades, os seus atributos e as relações entre elas, com base nos requisitos funcionais e operacionais definidos para a aplicação.

Após a definição inicial do modelo conceptual, procedeu-se à sua conversão para um modelo relacional, adequado à implementação numa base de dados Microsoft SQL Server [10], pelo seu elevado desempenho e conhecimento prévio dos intervenientes. Esta conversão incluiu a definição de chaves primárias e estrangeiras, bem como a aplicação de restrições de integridade e validação de dados. Esta base de dados possibilita o uso de ferramentas, como SQL Server Management Studio [11], que facilita a administração e criação do modelo lógico.

3.2 Estrutura do modelo

O modelo de dados organiza-se em cinco grandes grupos:

- Estrutura hierárquica;
- Zonas de Concentração e Apoio à População (ZCAP);
- Incidentes;
- Pessoas;
- Utilizadores.

A estrutura do modelo de dados apresentada nas sub-secções seguintes é usada quer no *backend* quer no *frontend*. A diferença entre ambos é a existência de dois campos adicionais na base de dados do *frontend* um "id" e um booleano "isSynced", que indicam respetivamente, a chave primária local gerada no dispositivo e o estado de sincronização do registo com o servidor. Estes campos são essenciais para suportar as operações *offline-first* da aplicação e a posterior reconciliação de dados com o *backend* durante o processo de sincronização. A chave primária do *backend* é armazenada no *frontend* com a designação "remoteId".

A generalidade das tabelas, tem como campos comuns:

- *startDate* - data de início da validade, de preenchimento obrigatório.
- *endDate* - data de fim da validade, de preenchimento opcional.
- *createdAt* - data e hora em que o registo foi criado localmente ou no servidor.
- *lastUpdatedAt* - data e hora da última modificação do registo, usada para deteção de alterações durante a sincronização.

Os campos *startDate* e *endDate* são utilizados para evitar a remoção definitiva de registos da base de dados. Desta forma, é possível manter um histórico completo e aceder a registos anteriores, mesmo após deixarem de estar ativos, permitindo uma fácil procura e análise retrospectiva dos dados. Estes existem também pelo facto de que certas entidades ou tipos de categorias podem mudar de nome e, assim, o registo do nome antigo continua na base de dados para futuro acesso.

3.2.1 Estrutura Hierárquica

A estrutura hierárquica serve para armazenar dados relacionados com as divisões administrativas, sendo esta elaborada de forma genérica. É possível armazenar o nível de cada divisão administrativa, uma divisão administrativa e ainda guardar detalhes sobre a mesma.

Para o armazenamento de níveis administrativos é utilizada a tabela *treeLevels* representada na Figura 3.1. Esta visa guardar o nível e nome de cada nível de divisão administrativa, sendo 1 o maior nível.

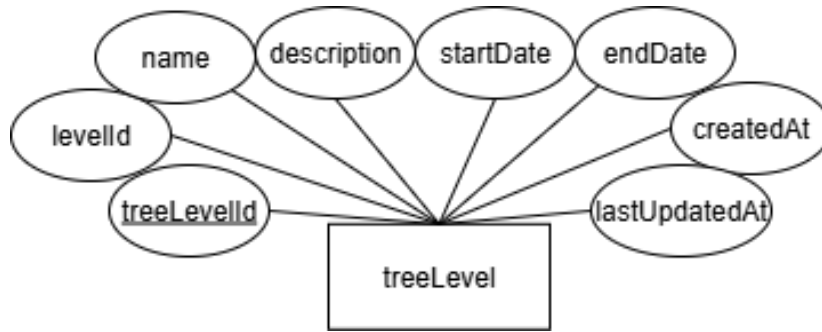


Figura 3.1: Representação do modelo do nível das divisões administrativas.

Cada elemento da estrutura hierárquica é armazenado na tabela *tree* representada na Figura 3.2. Esta tabela visa relacionar o nome de uma região com o seu respetivo nível (*treeLevelId* representado na Figura 3.1) e, se esta não tiver o nível mais alto estabelecido, relacionar com uma região pai, sendo este outro elemento da estrutura.

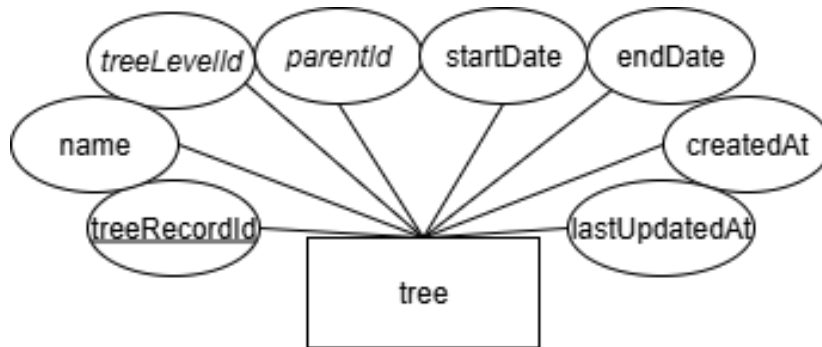


Figura 3.2: Representação do modelo dos elementos da estrutura.

A tabela *treeRecordDetailTypes*, representada na Figura 3.3, é usada para armazenar, previamente, todos os tipos de detalhes que podem ser associados a um elemento da estrutura, de forma genérica. Este armazena o nome do tipo e em que unidade deve ser lido (inteiro, *string*, *boolean*...).

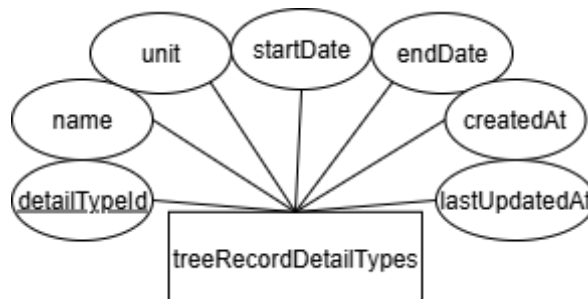


Figura 3.3: Representação do modelo dos tipos de detalhes.

Todos os detalhes de cada elemento da estrutura são armazenados na tabela *treeRecordDetails* representada na Figura 3.4. Esta armazena o valor do detalhe, o tipo de detalhe

(Figura 3.3) e a que elemento da estrutura hierárquica pertence.

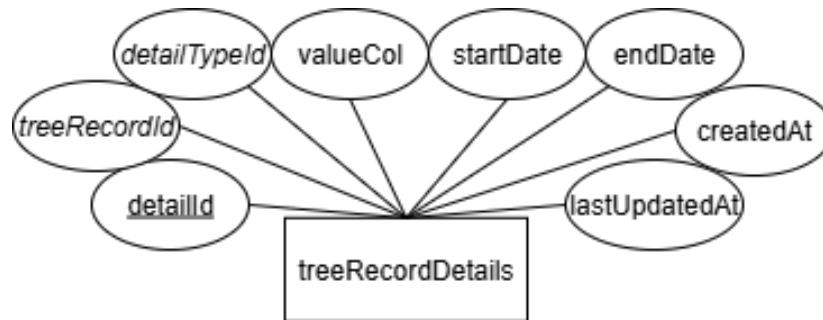


Figura 3.4: Representação do modelo dos detalhes.

A tabela *treeLevelDetailType*, representada na Figura 3.5, tem como objetivo associar informação entre um tipo de detalhe (Figura 3.3) e um nível da divisão administrativa (Figura 3.1). Esta associação serve para saber que tipos de detalhes podem descrever um certo nível, como por exemplo, um tipo de detalhe sobre um código de um país apenas poderá ser usado para detalhar um elemento da estrutura hierárquica com o nível de país.

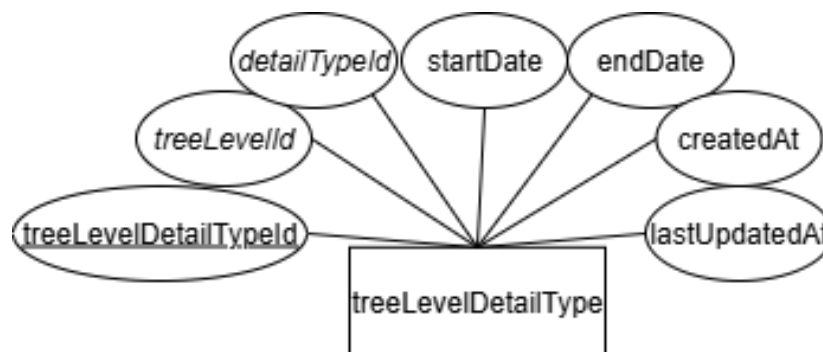


Figura 3.5: Representação do modelo dos detalhes.

3.2.2 Pessoas

O modelo das pessoas serve para o registo e gestão de todos os indivíduos ou famílias que, devido a uma emergência ou desastre grave, necessitam ou necessitaram de frequentar uma ZCAP.

Antes de armazenar pessoas, é necessário a existência de uma tabela que armazene todos os destinos de partida de uma ZCAP. Para isso, é utilizada a tabela *departureDestination* que armazena todos os tipos de destino aceites pelo sistema, representada na Figura 3.6.

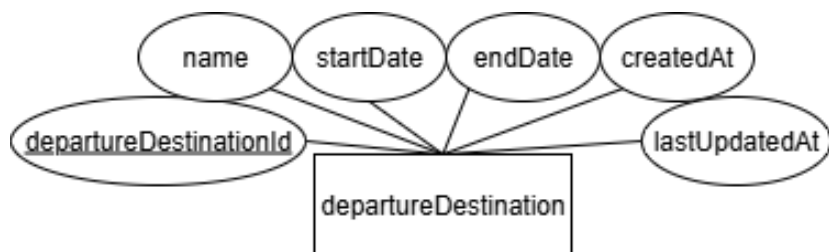


Figura 3.6: Representação do modelo dos destinos de partida de uma ZCAP.

No armazenamento e gestão das informações de uma pessoa é possível guardar informações quanto ao nome, idade, contacto, local de residência (elemento da estrutura hierárquica representado na Figura 3.2) e data de entrada. Estas informações serão armazenadas numa primeira fase de entrada na ZCAP. Numa segunda fase é possível armazenar informação como data de saída, data de aniversário, nacionalidade (chave estrangeira assim como o código do país), morada, número de identificação de segurança social (NISS), destino de saída da ZCAP (Figura 3.6) e o contacto desse mesmo destino. Diferentemente de muitas outras tabelas, esta não tem registo de *startDate* e *endDate* uma vez que saberemos que esta só estará ativa enquanto não existir registo da data de saída da ZCAP (*departureDateTime*). Assim, caso a mesma pessoa precise de abrigo em incidentes diferentes, acabará por existir dois registos dessa mesma pessoa. Estas informações são armazenadas na tabela *persons* como demonstrado na Figura 3.7.

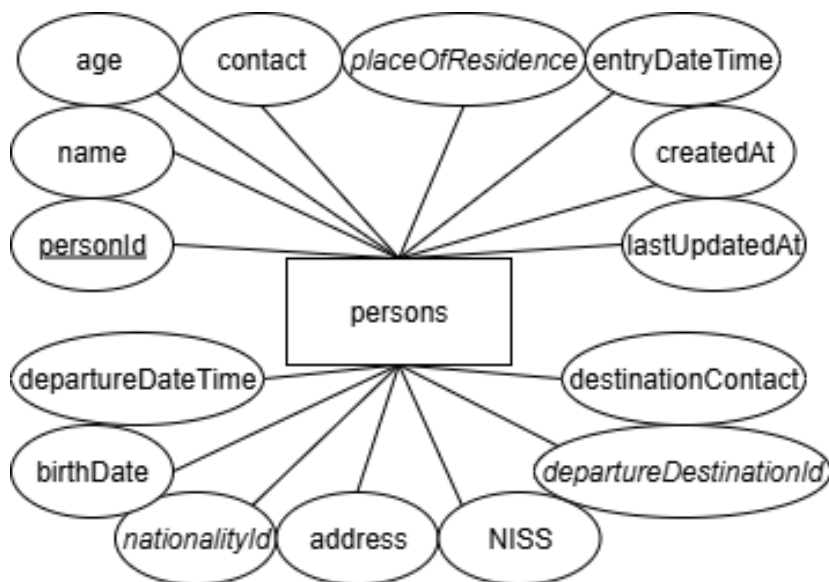


Figura 3.7: Representação do modelo das pessoas.

As pessoas podem ter necessidades especiais (medicamentos, gravidez, ...) ou necessidade de algum tipo de suporte (alojamento, comida, vestuário, ...). As necessidades especiais disponíveis no sistema são armazenadas na tabela *specialNeeds*, onde se encontra apenas o nome da necessidade especial. Na tabela *personSpecialNeeds* é onde a relação da pessoa

com a necessidade especial é associada. Uma pessoa pode ter várias necessidades especiais e várias pessoas podem ter a mesma necessidade especial. Para a necessidade de suporte é exatamente o mesmo que a necessidade especial. Existe uma tabela *supportNeeded* que armazena todos os tipos de necessidades de suporte aceites pelo sistema e este é associado à pessoa pela tabela *personSupportNeeded*. Uma vez que as tabelas são muito idênticas, para demonstração, apenas é usada a necessidade especial e a relação desta com uma pessoa nas Figuras 3.8 e 3.9, respetivamente.

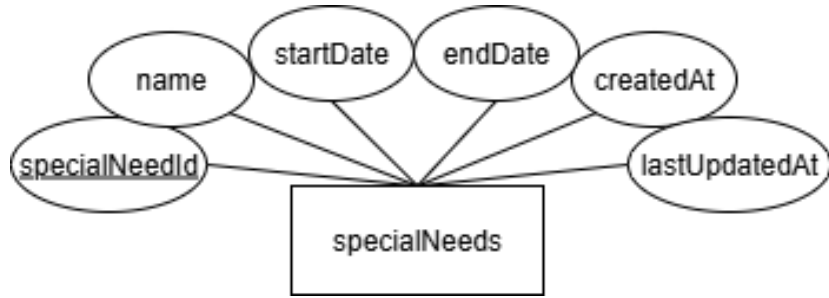


Figura 3.8: Representação do modelo das necessidades especiais.

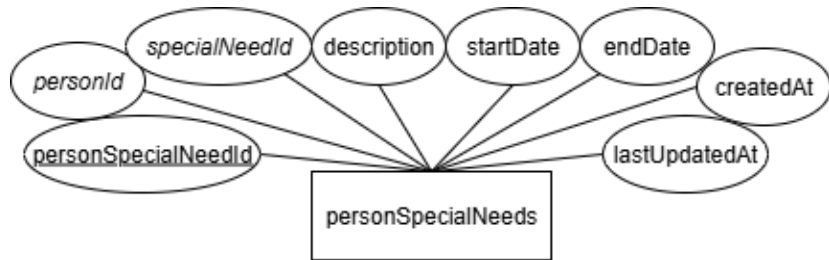


Figura 3.9: Representação do modelo da relação da necessidade especial com uma pessoa.

Por fim, os indivíduos que necessitam de frequentar uma ZCAP, muitas vezes têm familiares. Para armazenamento e gestão dessa informação, o sistema admite os tipos de relação (pai, mãe, primo, tia, ...) de parentesco na tabela *relationType*, representada na Figura 3.10. Para associação do tipo de relação com as pessoas, é utilizada a tabela *relation*, representada na Figura 3.11. Nesta tabela é representado o tipo de relação que a pessoa 1 tem com a pessoa 2, sendo estas chaves estrangeiras de pessoas.

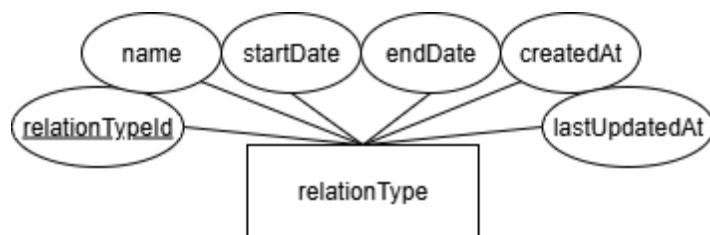


Figura 3.10: Representação do modelo do tipo de relações de parentesco.

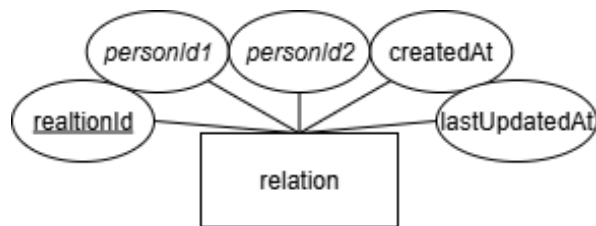


Figura 3.11: Representação do modelo de graus de parentesco.

3.2.3 Utilizadores

O modelo de dados dos utilizadores serve para armazenar e gerir as informações de todos os colaboradores, direta ou indiretamente, das ZCAPs. Para cada utilizador, é necessário haver registo de suas permissões dentro do sistema.

Para regular as permissões por regiões é necessário armazenar um perfil de dados do utilizador, representado na tabela *userDataProfiles* na Figura 3.12. Este guarda apenas o nome do perfil.

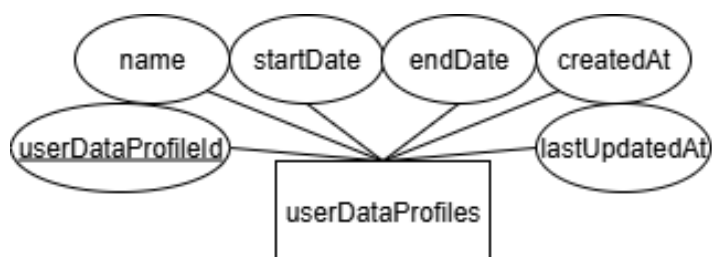


Figura 3.12: Representação do modelo das permissões por região aceites pelo sistema.

Para realizar a associação do perfil com as regiões a quais o utilizador tem acesso é necessário criar e armazenar um detalhe de perfil de dados do utilizador na tabela *userDataProfileDetails*, representado na Figura 3.13. A informação armazenada contém um perfil de dados do utilizador (Figura 3.12) e a que zona territorial (mencionada na Subseção 3.2.1) que o perfil tem acesso. Esta não contém campos como *startDate*, *endDate*, *createdAt* e *updatedAt* pois desempenha, apenas, um papel estrutural e associativo, uma vez que a validade é herdada da própria entidade *userDataProfiles*.

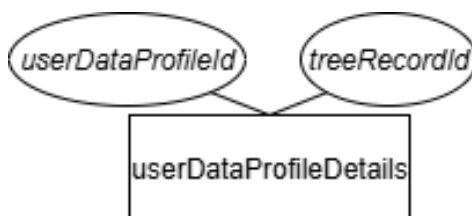


Figura 3.13: Representação do modelo da associação da região territorial com os perfis de permissões de dados.

Na tabela *userProfiles* são armazenados todos os perfis disponíveis no sistema, como

ilustrado na Figura 3.14. Aqui é armazenado o nome do perfil de utilizador. Estes perfis serão utilizados no sistema para determinar as funcionalidades disponíveis ao utilizador.

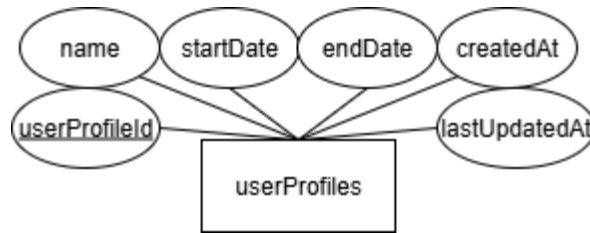


Figura 3.14: Representação do modelo dos perfis de utilizador.

Para as permissões de visualização de ecrãs dentro do *frontend* é necessário associar cada perfil às permissões dentro da aplicação. Estas permissões são geridas com chaves, sendo cada chave um acesso a um ecrã diferente. Estas chaves são armazenadas na tabela *userProfileAccessKeys*, como demonstra a Figura 3.15. Esta tabela apenas armazena as chaves de cada ecrã do sistema.

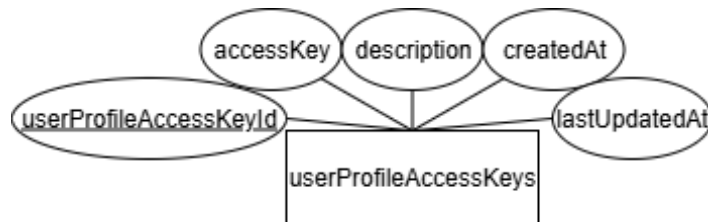


Figura 3.15: Representação do modelo das chaves de acesso.

Para a associação destas chaves com os perfis, é utilizada a tabela *userProfileAccessAllowance*, que armazena um perfil de utilizador (Figura 3.14), uma chave de acesso (Figura 3.15) e o tipo de acesso (leitura, leitura/escrita ou sem acesso). A tabela *userProfileAccessAllowance* encontra-se ilustrada na Figura 3.16.

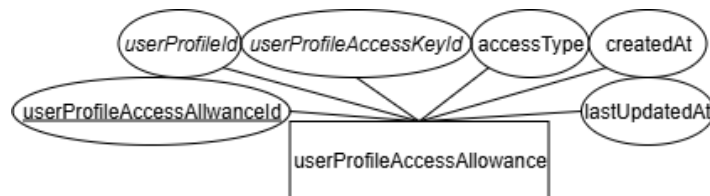


Figura 3.16: Representação do modelo da associação chaves de acesso-perfil de utilizador.

Por fim, no armazenamento de um utilizador no sistema (tabela *users* representada na Figura 3.17), é possível armazenar nome, nome de utilizador, senha, perfil de utilizador (Figura 3.14), permitindo saber as permissões dentro da aplicação, e perfil de dados do utilizador (Figura 3.12), para saber a que regiões tem acesso.

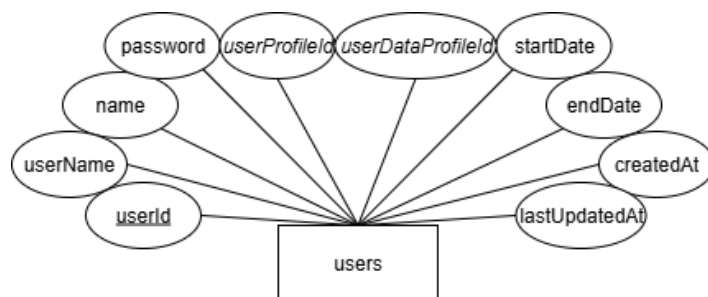


Figura 3.17: Representação do modelo dos utilizadores.

3.2.4 ZCAPs

Para gerir uma ZCAP, é necessário que existam entidades competentes. Para esse efeito, foram criadas as tabelas *entityTypes* (Figura 3.18) e *entities* (Figura 3.19). Estas armazenam os tipos de entidades aceites pelo sistema, guardando apenas o nome desse tipo (ex.: Bombeiros, Polícia de Segurança Pública, etc.), e as próprias entidades, que armazenam o nome da mesma, a que tipo estão associadas, pelo menos um contacto e, opcionalmente, o email e outro contacto associado à entidade.

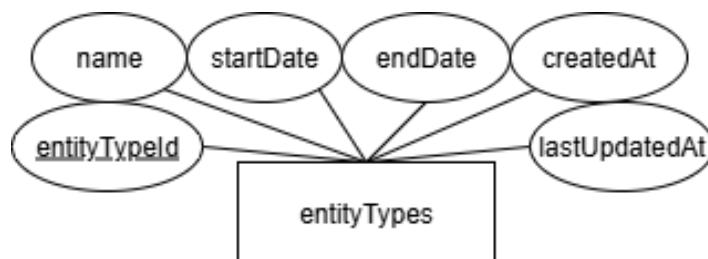


Figura 3.18: Representação do modelo tipo de entidades.

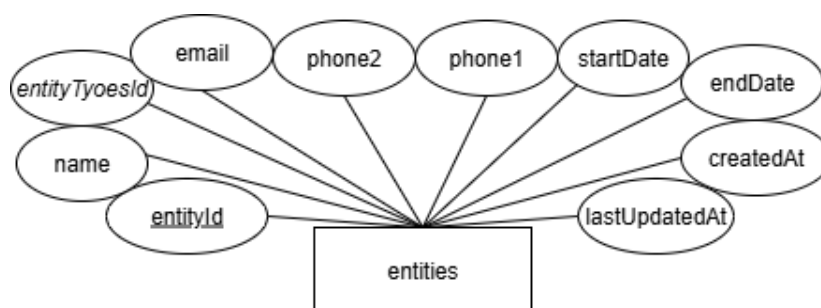


Figura 3.19: Representação do modelo das entidades.

Antes do armazenamento da ZCAP, é necessário existir a tabela *buildingTypes*, que armazena os tipos de edifício aceites pelo sistema (escola, pavilhão, etc.). Esta apenas armazena o nome do tipo de edifício e encontra-se representada na Figura 3.20.

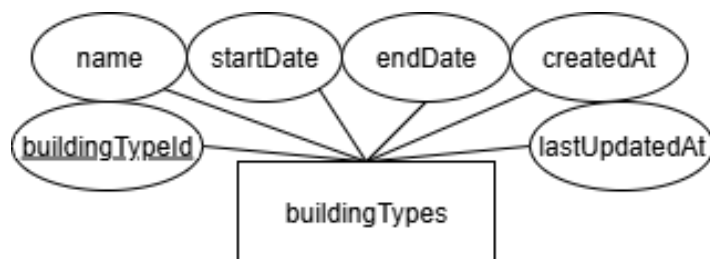


Figura 3.20: Representação do modelo dos tipos de edifícios.

As informações gerais de uma ZCAP são armazenadas na tabela *zcaps*, representada na Figura 3.21. Esta tem informações como nome, tipo de edifício (Figura 3.20), morada, região na estrutura hierárquica (sub-seção 3.2.1) e entidade responsável. É possível ainda armazenar valores de latitude e longitude, embora não sejam obrigatórios.

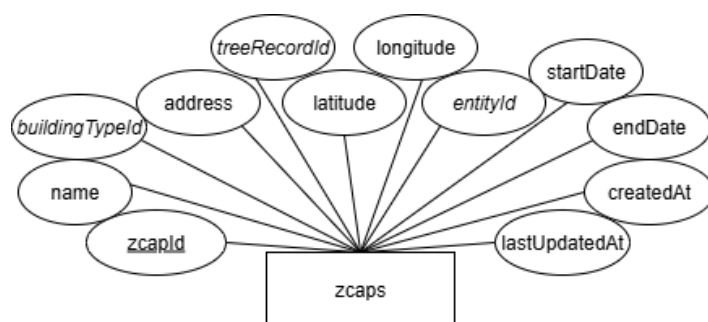


Figura 3.21: Representação do modelo das ZCAPs.

Cada ZCAP pode ter vários detalhes, que definem as suas características. Estes armazenam a chave estrangeira da ZCAP a que pertence, o tipo de detalhe (Figura 3.23) e o valor do detalhe. Os detalhes estão representados na Figura 3.24.

Os tipos de detalhes são armazenados na tabela *zcapDetailTypes*, representada na Figura 3.23. Estes armazenam todos os tipos de detalhes aceites pelo sistema, guardando o seu nome, a sua categoria (Figura 3.22), o tipo do dado a ser armazenado (neste caso é apenas um enumerado de tipos, tanto no *backend*, como no *frontend*) e se esse tipo de detalhe é de preenchimento obrigatório para o funcionamento da ZCAP.

Por fim, no armazenamento dos detalhes, é utilizada a tabela *zcapDetails* (Figura 3.24) que armazena o identificador da ZCAP a que pertence (Figura 3.21), o identificador do tipo de detalhe (Figura 3.23) e o valor do detalhe, que irá ser lido de acordo com o tipo do dado mencionado na tabela *zcapDetailTypes*.

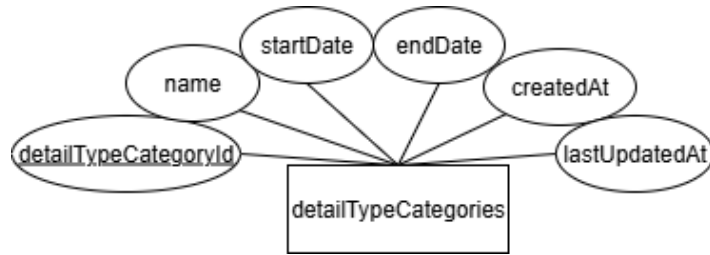


Figura 3.22: Representação do modelo das categorias de detalhe das ZCAPs.

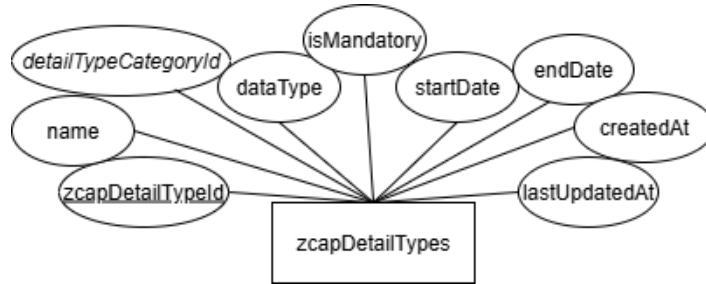


Figura 3.23: Representação do modelo dos tipos de detalhe das ZCAPs.

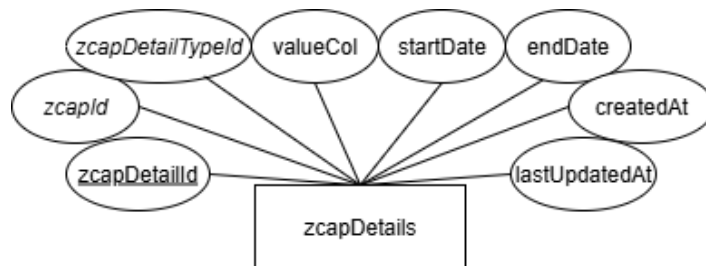


Figura 3.24: Representação do modelo dos detalhes das ZCAPs.

3.2.5 Incidentes

O modelo dos incidentes serve para armazenar informações sobre todos os acidentes graves ou catástrofes que existem.

O sistema só aceita os tipos de incidentes armazenados na tabela *incidentTypes*, representada na Figura 3.25. Este armazena o nome de todos os tipos de incidentes existentes no sistema (Terramoto, Pandemia, etc.).

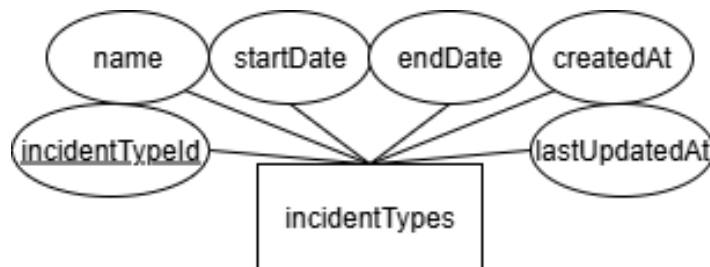


Figura 3.25: Representação do modelo dos tipos de incidentes.

A tabela *incidents* armazena cada incidente existente. Este armazena o tipo de incidente, em que zona este acontecimento teve impacto (campo *treeRecordId* que refere a um elemento da árvore) e o período de ativação. Os incidentes estão representados na Figura 3.26

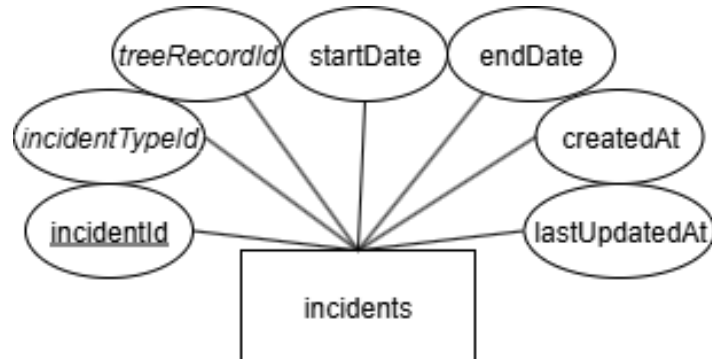


Figura 3.26: Representação do modelo dos incidentes.

A tabela *incidentZcaps* armazena a associação de um incidente com uma ZCAP e a entidade responsável (ambas referidas na Subseção 3.2.4). Esta encontra-se representada na Figura 3.27. Isto permite saber qual ou quais incidentes originaram a abertura da respetiva ZCAP.

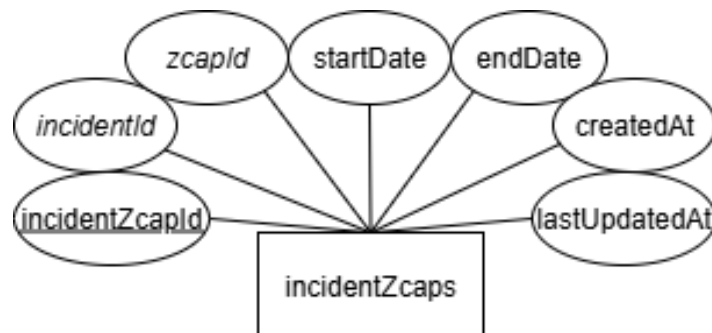


Figura 3.27: Representação do modelo da associação incidente-ZCAP.

Na tabela *incidentZcapPersons* é associada uma entrada da tabela *incidentZcaps* com uma pessoa. Assim, facilmente conseguimos obter informação de qual ZCAP em que a pessoa está e o porquê dessa ZCAP estar ativa (incidente).

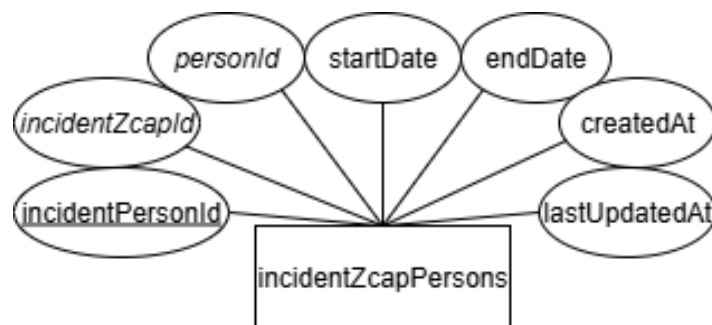


Figura 3.28: Representação do modelo da associação incidente-ZCAP-pessoas.

3.3 Extensibilidade e Escalabilidade

O modelo de dados foi concebido com foco na extensibilidade e escalabilidade, de forma a garantir a sua adaptabilidade a novos requisitos e a sua capacidade de suportar volumes crescentes de informação sem degradação significativa de desempenho.

Em termos de extensibilidade, a estrutura foi organizada de forma modular, com separação clara entre diferentes domínios de dados, como utilizadores, zonas territoriais (árvore hierárquica), ZCAPs, pessoas e incidentes. Esta organização facilita a introdução de novas entidades, atributos ou relações, sem a necessidade de alterações disruptivas nas tabelas existentes. Por exemplo, a inclusão futura de novos tipos de incidentes, edifícios, utilizadores, relações ou níveis hierárquicos poderá ser efetuada com alterações mínimas ao modelo atual.

A utilização de chaves estrangeiras bem definidas e de relações normalizadas até à terceira forma normal contribui também para a robustez do modelo face a alterações e ampliações, promovendo a consistência dos dados e a facilidade de manutenção.

Quanto à escalabilidade, o modelo foi implementado sobre um sistema de gestão de base de dados relacional (Microsoft SQL Server), que oferece mecanismos eficientes de indexação, consulta e gestão de dados em larga escala. Foram consideradas técnicas como a utilização de índices em campos de pesquisa frequente e a separação lógica de responsabilidades entre tabelas, o que permite otimizar o desempenho em cenários com elevado volume de dados ou múltiplos utilizadores em simultâneo.

Capítulo 4

Camada de apresentação - *Frontend*

Este capítulo descreve o desenvolvimento da camada de apresentação da aplicação, ou seja, a interface com a qual os utilizadores interagem. É primeiramente apresentada a tecnologia selecionada para a implementação (*Flutter*) e as razões que motivaram essa escolha.

Na segunda secção é descrita a estrutura do projeto, as suas dependências, a escolha do modo *offline-first* e os mecanismos de persistência local de dados.

Na terceira e última secção, detalham-se as principais funcionalidades desenvolvidas, como o acesso à aplicação, o suporte multi-língua, a estrutura hierárquica em árvore, a gestão das ZCAPS, Incidentes e Pessoas, a Gestão de Utilizadores que por sua vez se divide em Perfis e Acesso a Dados.

Em algumas dessas funcionalidades, são incluídos fluxogramas ilustrativos que evidenciam o fluxo dos processos.

Por fim, ainda nesta última secção, é abordado o processo de sincronização de dados, complementado com um fluxograma que demonstra de forma visual o seu funcionamento.

4.1 Introdução

Para o *frontend* designadamente a aplicação para Windows Desktop foi selecionada a *framework* **Flutter** pelas suas características de desempenho e capacidade de suportar multi-plataformas a partir de um único código-base. Apesar de ter sido criada pela Google para desenvolvimento de aplicações Android e iOS, Flutter suporta de igual forma o desenvolvimento de aplicações Desktop para Windows, Linux ou MacOS. O seu ecossistema de bibliotecas é vasto e atualizado existindo uma comunidade ativa e em crescimento. A escolha inicial foi o React Native e não Flutter por ter eficiência e desempenho semelhante e a prévia experiência com JavaScript facilitaria o ritmo do desenvolvimento. No entanto, após algum esforço percebeu-se que o suporte para Desktop e, especial para o armazenamento local não tem ainda bibliotecas que permitissem em tempo útil a conclusão do projeto, pelo que se alterou a escolha para o Flutter. Foi também avaliada a opção Kotlin Multiplatform tendo

sido descartada pela performance relativa ao Flutter.

4.2 Estrutura do Projeto

O projeto segue uma organização modular e escalável, baseada numa estrutura por *features*, que promove a separação de responsabilidades e facilita a manutenção e evolução da aplicação. A estrutura geral de pastas é a seguinte:

```
assets/  
  config/  
  images/  
  translations/  
lib/  
  core/  
    services/  
    utils/  
  features/  
    home/  
      screens/  
    login/  
    settings/  
      entityTypes/  
      entities/  
      treeLevels/  
      tree/  
      users/  
      ...  
  shared/  
  widgets/
```

As pastas refletem a separação de responsabilidades com o seguinte objetivo:

- **assets/-** ficheiros imagem ou de texto para utilização durante a execução da aplicação.
- **core/-** contém elementos transversais à aplicação, como definições globais (*constants*), modelos de dados genéricos (*models*), serviços de acesso a dados e sincronização (*services*) e utilitários (*utils*).
- **features/-** organiza o código por funcionalidade (*home*, *login*, *settings*, ...), cada uma com o seu próprio modelo de dados (*model/*) e ecrãs (*screen/*).
- **shared/-** contém elementos partilhados entre diferentes funcionalidades.

- **widgets/-** contém elementos de interface personalizados, reutilizáveis, como botões, *date pickers*, *list views*, *snack bars*, etc por forma a manter o aspeto e funcionalidade consistente em todos os ecrãs de funcionalidades.

4.2.1 Dependências Principais

A aplicação recorre a várias dependências para suportar funcionalidades específicas, nomeadamente:

- **bCrypt** - disponibiliza funções de criptografia (*hashing* e verificação de *passwords*).
- **connectivity_plus** - permite a utilização de funções de verificação de conectividade da rede.
- **cupertino_icons** - disponibiliza os ícones compatíveis com o estilo iOS.
- **dropdown_search** - permite criar campos de lista de seleção avançados.
- **http** - usado para realizar as chamadas HTTP à API de *backend*.
- **intl** - fornece suporte à formatação de datas, horas e outros elementos de internacionalização.
- **isar** e **isar_flutter_libs** - base de dados NoSQL para a persistência de dados *offline*.
- **jwt_decoder** - contém as funcionalidades necessárias para decodificar e validar os tokens JWT recebidos da API.
- **path_provider** - para aceder a diretórios do sistema de ficheiros.
- **provider** - para a gestão de estado reativa da aplicação.
- **shared_preferences** - para armazenar dados simples de forma persistente (ex. dimensão da janela na última utilização ou o *darkMode*).
- **window_size** - usado para gerir o tamanho e posicionamento da janela na versão *desktop*. Em dispositivos móveis o tamanho da janela não é adaptável e não causa problemas de posicionamento dos componentes mas nas versões Desktop podem surgir comportamentos indesejados pelo que se limitou a dimensão mínima da janela.

4.2.2 Desenvolvimento

O processo de desenvolvimento é coadjuvado por ferramentas como o **build_runner** e o **isar_generator**, principalmente para a geração do código associado aos modelos e esquemas de dados da base de dados Isar.

4.2.3 Modo *Offline-First* e persistência de Dados

Um dos objetivos principais da aplicação é a capacidade de operar na forma **offline-first**, permitindo aos utilizadores aceder e registar informações mesmo na ausência de ligação ao *backend*. Para que isso aconteça, os dados são armazenados localmente utilizando a base de dados **Isar** [8], uma base de dados NoSQL embutida desenvolvida especificamente para aplicações Flutter, oferecendo alta performance, simplicidade de uso e integração nativa com o ecossistema Flutter. A informação armazenada nesta base de dados é posteriormente sincronizada com o servidor através do serviço de sincronização que recorre à Web API (se e quando esta estiver disponível), garantindo a integridade e consistência de forma global.

4.3 A Aplicação

A unidade central do sistema são as ZCAPS e sua função, no entanto um conjunto de outras funcionalidades foram necessárias para as implementar de uma forma lógica e organizada. Descrevem-se agora estas funcionalidades e o detalhe da sua implementação.

4.3.1 Acesso à aplicação

O acesso à aplicação é feito por intermédio de um utilizador e palavra-passe podendo neste ponto seleccionar ou modificar o idioma a utilizar (ver secção 4.3.2).

Apesar da aplicação estar desenhada para funcionar no modo *offline-first*, o processo de *login* começa pela tentativa de obtenção de um *token* válido pela API para poder executar as operações de sincronização de dados. Quando esta tentativa falha é que valida localmente as credenciais. Caso a autenticação *online* seja bem sucedida, é verificado se localmente já existe informação do utilizador e em caso negativo é forçado o processo de sincronização pois a atribuição de acessos (ver secção 4.3.7) é validada com os dados locais.

O fluxo de validação do acesso obedece ao descrito na Figura 4.1.

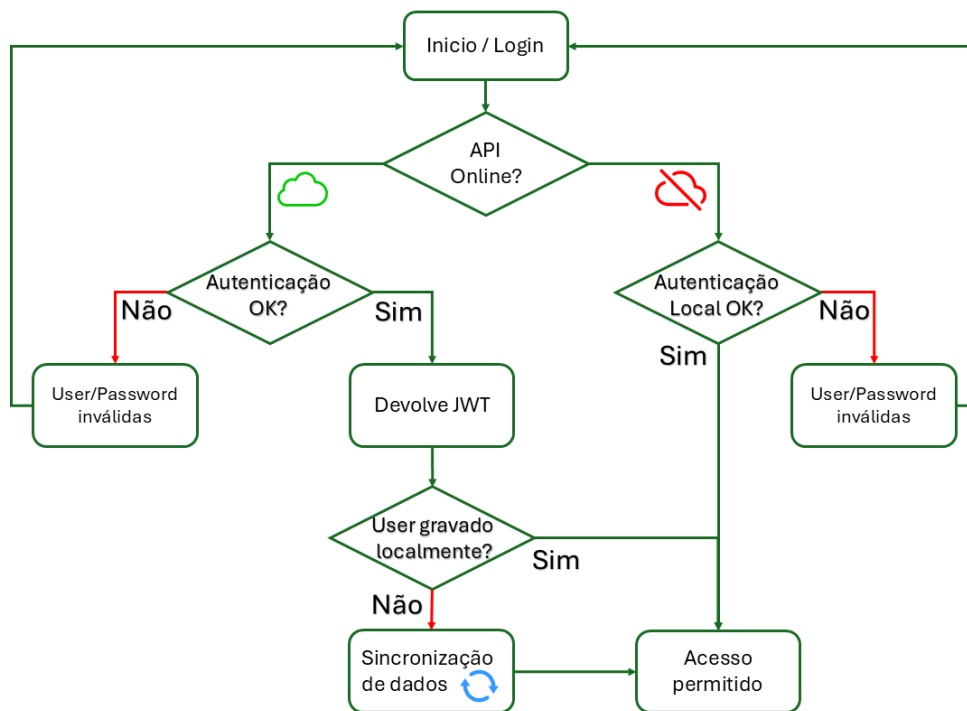


Figura 4.1: Fluxograma de acesso à aplicação

4.3.2 Multi-língua

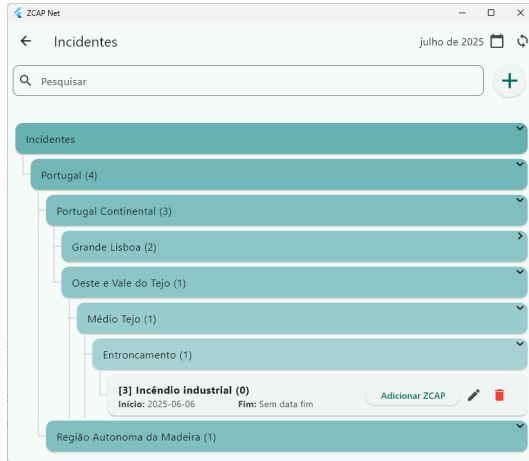
A aplicação suporta múltiplos idiomas, permitindo ao utilizador seleccionar o idioma no momento do *login*. Os idiomas disponíveis estão listados no ficheiro `languages.json`, localizado na pasta `assets/translations`. Para cada idioma existe no mesmo diretório, com a descrição do campo `code` do ficheiro de definição, um ficheiro com os pares *chave-valor*.

A abordagem utilizada para desenvolvimento desta funcionalidade foi inspirada no formato utilizado pelo sistema Android, mas usando o formato JSON em vez de XML.

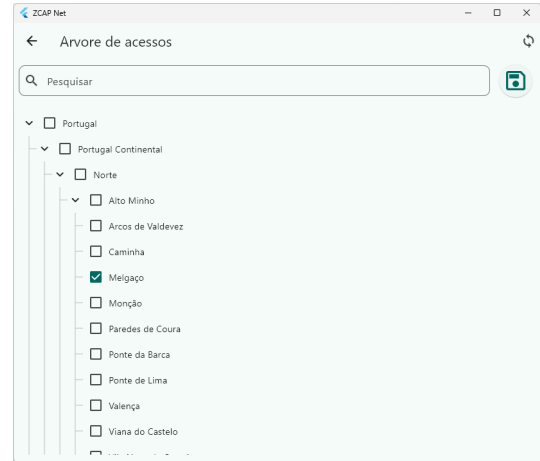
4.3.3 Árvore hierárquica

Da nossa análise, a lógica de gestão das ZCAPS é atualmente realizada por localização pelo que, para uma organização hierárquica por localizações optou-se por uma estrutura em árvore, conforme descrito no modelo de dados (ver secção 3.2.1), onde cada elemento da árvore tem um elemento pai (exceto o elemento *root*) e pode ter vários filhos. Desta forma, o número de níveis possíveis de utilizar são ilimitados permitindo uma gestão flexível quando e se for necessário ajustar a mesma. Esta organização ajudou também a simplificar o modelo de atribuição de acesso a dados (ver secção 4.3.7).

No desenvolvimento desta funcionalidade foi utilizada a biblioteca `animated.tree.view` [12] que oferece um interface dinâmico e interativo para visualização deste tipo de estruturas. Dois exemplos de utilização desta funcionalidade são visíveis na figura 4.2.



(a) Ecrã de Incidentes



(b) Ecrã de Gestão de acessos

Figura 4.2: Exemplos de uso da `animated_tree_view`

4.3.4 ZCAPS

A unidade central do sistema, ZCAPs, tem pontos de contacto com a maioria das restantes funcionalidades. Foi concebida de forma a poder adaptar-se ou readaptar-se às suas características quando necessário.

Uma ZCAP tem de necessariamente estar associada a um item da árvore hierárquica. Tem uma tipologia de edifício e uma lista de detalhes sendo que os detalhes são agrupados por categorias e cada detalhe pode ser de um tipo diferente de dados e ser de preenchimento obrigatório ou não (Figura 4.3).

O armazenamento dos dados do detalhe é sempre feito no formato *string* sendo usadas conversões de e para o formato correto. São feitas previamente validações nos campos de inserção dos valores, por forma a evitar erros de inserção de dados.

Uma ZCAP é também unidade fundamental no processo dos Incidentes pois é durante esse processo que as ZCAPs são utilizadas para o acolhimento de Pessoas (Figura 4.4).

4.3.5 Incidentes

A funcionalidade dos Incidentes é o processo terminal no que respeita à utilização operacional da aplicação sendo que um incidente tem associado um tipo de incidente (tabela), um intervalo de tempo (data de início e fim) e pode ver associadas uma ou várias ZCAPs. Por sua vez, conforme a figura 4.4, cada ZCAP recebe uma ou várias Pessoas (ver secção 4.3.6).

4.3.6 Pessoas

Proporcionar alojamento temporário e seguro aos cidadãos e famílias deslocados em virtude de um Incidente é o objetivo de uma ZCAP. Quando uma ZCAP é associada a um Incidente é com o objetivo de acolher Pessoas.

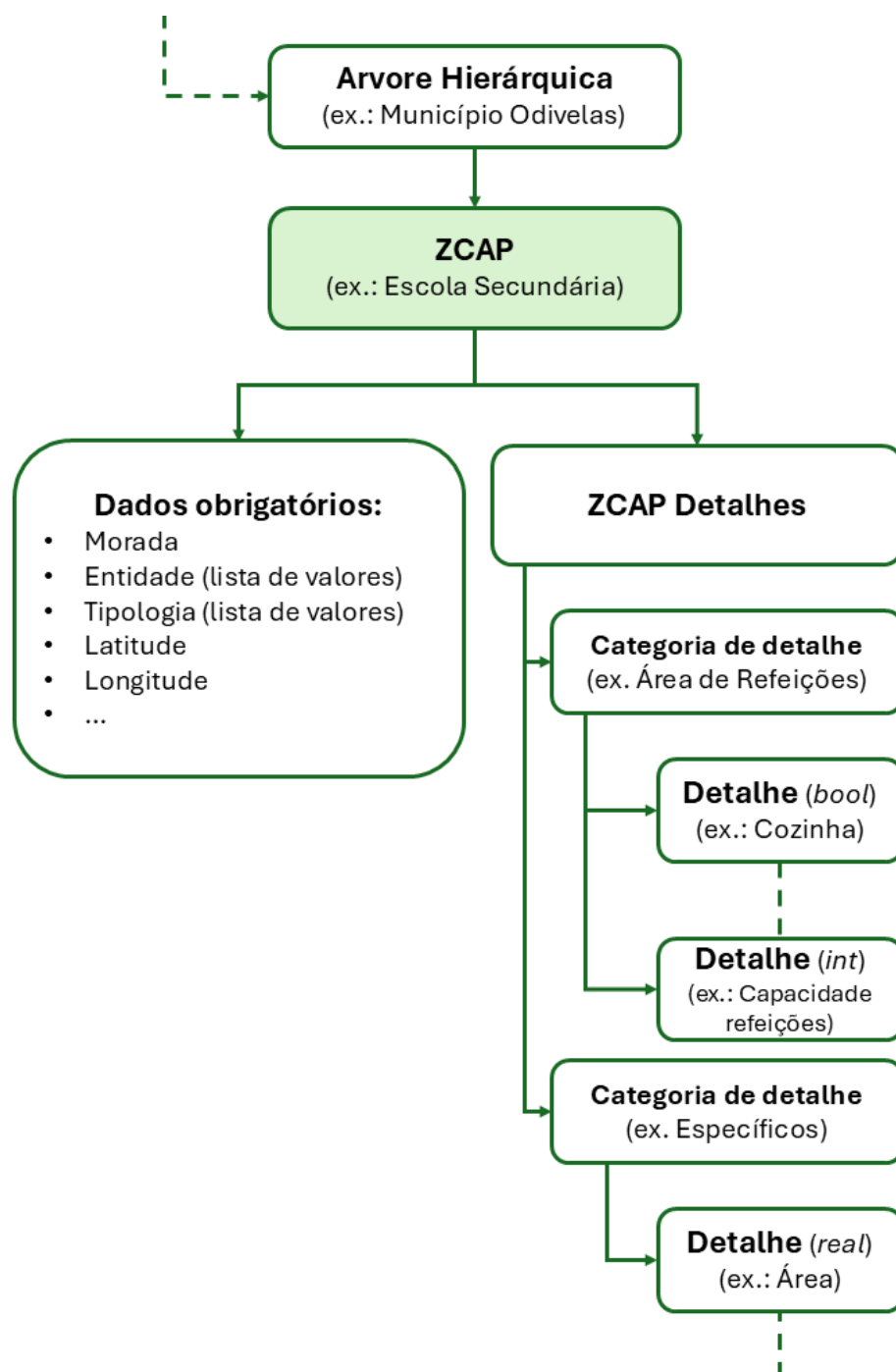


Figura 4.3: ZCAP - Modelo conceitual de uma ZCAP: associação à árvore e detalhe por categorias

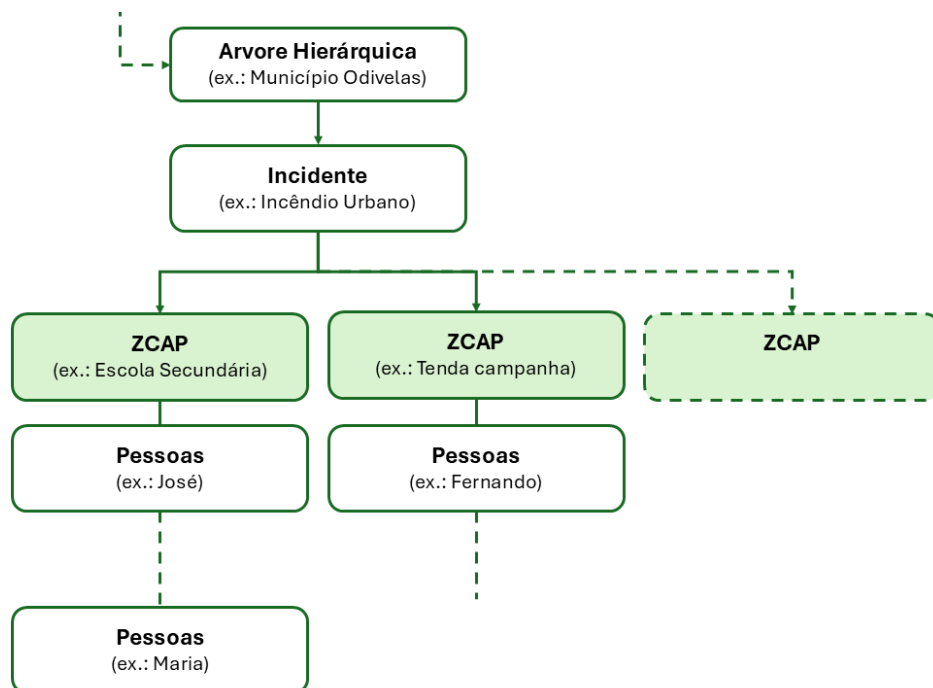


Figura 4.4: ZCAPs no contexto de um Incidente

Estas, passam por um processo inicial de acolhimento onde são recolhidos um conjunto mínimo de dados (obrigatórios) e depois por uma segunda fase onde se podem complementar dados de necessidades de apoio (lista de valores) ou necessidades especiais (lista de valores) para a Pessoa. No final é recolhida a data/hora e destino da Pessoa (ver secção 4.5).

4.3.7 Gestão de Utilizadores

O modelo de dados da aplicação foi concebido, desde início para considerar a necessidade de existência de utilizadores com níveis/perfis de acesso diferentes ao nível das funcionalidades assim como segregar o conteúdo nelas contido ou seja, cada utilizador para além dos habituais *userName* e *password* tem também associado um Perfil e um Acesso a Dados. É também possível determinar um período de acesso definindo a data de início (obrigatório) e fim do utilizador.

Perfil

A opção **Perfil** permite identificar qual o nível de acesso a cada funcionalidade do sistema assim como a permissão dentro dessa funcionalidade (figura 4.6).

Para cada funcionalidade existe uma opção de perfil com as opções:

- **Leitura/Escrita** - Acesso total, pode criar, alterar e eliminar registos.
- **Leitura** - Lê informação mas não pode criar, alterar ou eliminar registos.
- **Sem acesso** - Opção não disponível para o utilizador.

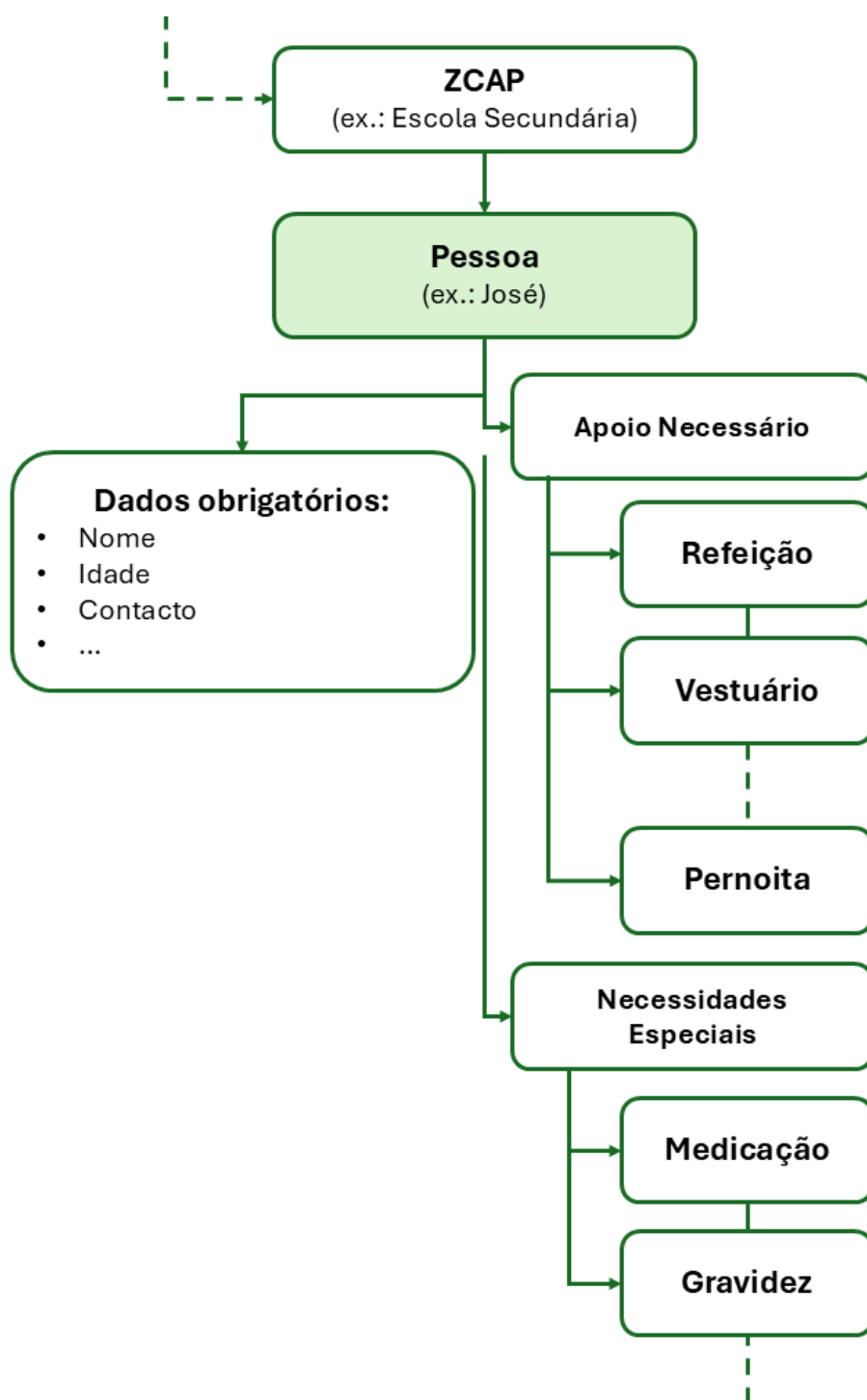


Figura 4.5: Pessoas - Esquema de utilização

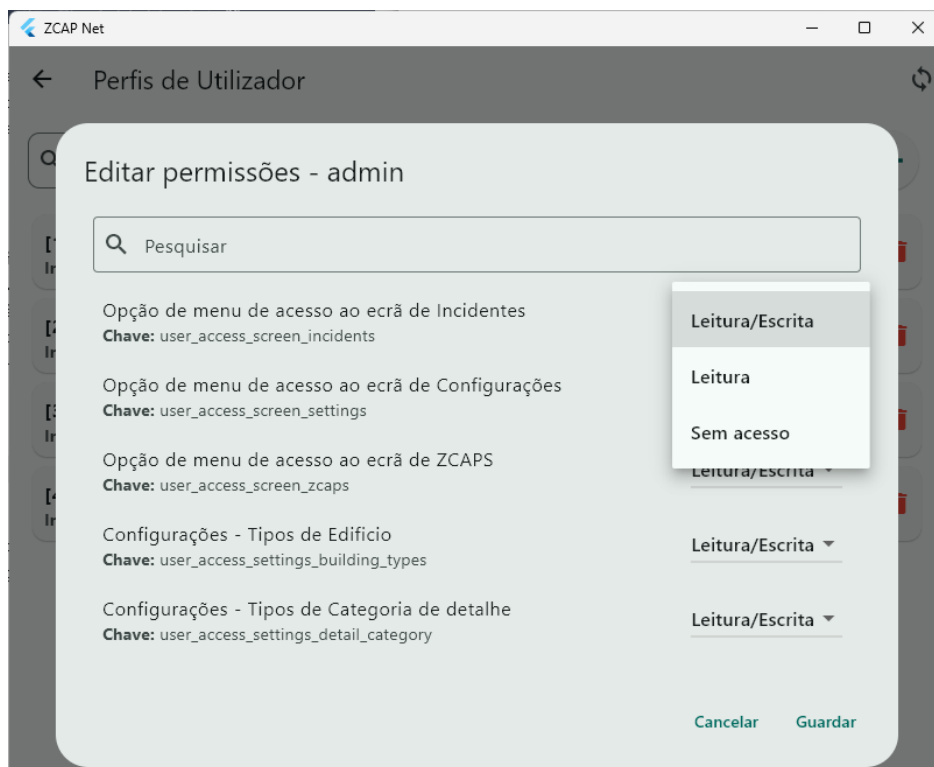


Figura 4.6: Perfil - Exemplo de permissões

Acesso a dados

Gerir o acesso às funcionalidades do sistema não é suficiente quando se suporta gestão de dados a uma escala considerável (ex.: Nacional).

É necessário também garantir a filtragem/segregação dos dados acessíveis a cada utilizador assim, cada utilizador tem associado um perfil de **Acesso a Dados** que define qual ou quais os dados associados à **Árvore Hierárquica** aos quais acede ou visualiza.

A figura 4.2b é demonstrativa deste exemplo onde o utilizador associado a este perfil só terá acesso ao nó ou lista de nós selecionados da árvore.

4.3.8 Sincronização

Tendo sido desenvolvida a aplicação no modo *offline-first*, era necessário criar um processo que permitisse enviar e receber a informação de e para a base de dados central.

Este, ocorre de forma automática a cada n segundos, conforme definido no ficheiro de configuração (`assets/config/config.json`) ou despoletado manualmente em qualquer dos ecrãs da aplicação.

O processo foi desenvolvido recorrendo às classes abstratas previamente criadas a quando da conceção do modelo de dados, onde já se previu esta necessidade. O processo utiliza as classes `IsarTable` e `ApiTable` sendo que para cada tabela da base de dados central e local existe uma classe `IsarTable` e `ApiTable` (exceção feita à entidade fraca dos **Acessos a dados**).

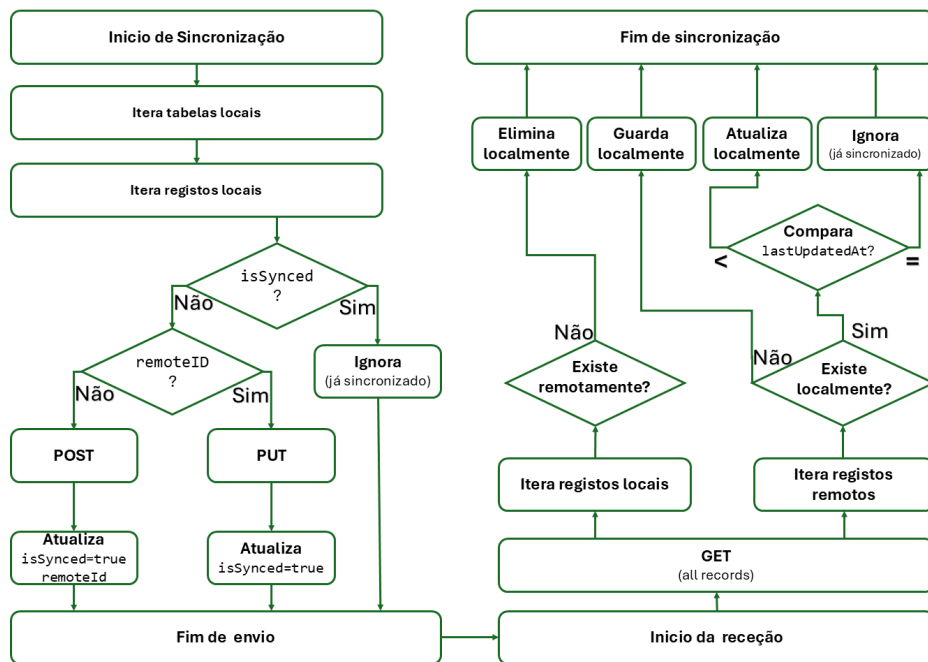


Figura 4.7: Fluxo de Sincronização de dados

O processo de sincronização utiliza o valor do campo `isSynced` para criar a lista de registros a sincronizar com a API. A inexistência de valor no campo `remoteId` determina se é um registro novo e é executada a operação POST do serviço dedicado a transações com a API para a base de dados central ou uma atualização através da operação PUT do mesmo.

Existe a possibilidade de estar a enviar um registro que foi também alterado por um outro utilizador. Não foi previsto neste projeto a existência de um "gestor de conflitos" estando referenciado como **Trabalho Futuro**. Para estes casos, é o *backend* que resolve estas divergências usando para isso o `lastUpdatedAt` e selecionando o mais recente.

Capítulo 5

Camada de Processamento - *Backend*

O *backend* consiste em desenvolver uma *Web API* que utiliza *Spring framework* [13], principalmente o módulo *Spring Web MVC* [14] e a extensão *Spring Boot* [15], em conjunto com a linguagem Kotlin [16], uma vez que é a linguagem mais conhecida pelos intervenientes.

No mapeamento das entidades para uma base de dados relacional, é usado Jakarta JPA [17].

Na ajuda do armazenamento e gestão na base de dados, é utilizado Spring Data JPA [18], que fornece uma camada de abstração por cima do JPA, facilitando muito o desenvolvimento.

Para serviços de autenticação e autorização, é usado JWT (*JSON Web Token*) [19], por ser um formato compacto e seguro de *tokens*.

A primeira secção deste capítulo (secção 5.1) descreve a estrutura e organização, assim como a função de cada um, para depois em sub-secções se descrever cada um deles individualmente. A segunda secção (secção 5.2) descreve o formato dos *endpoints* criados e o seu formato geral, quer em caso de resposta com sucesso (ver secção 5.3.1) quer em caso de erro (secção 5.3.2). A última secção deste capítulo (secção 5.4) é dedicada à documentação da Web API, onde se utilizou uma biblioteca conhecida, *Swagger* [20], para agilizar o processo de documentação.

5.1 Estrutura e Organização

A arquitetura do sistema foi dividida em camadas bem definidas: Controladores (*Controllers*), Serviços (*Services*), Repositórios (*Repositories*) e Entidades (*Entities*). A estrutura é a seguinte:

```
api/  
  exceptions/  
  login/  
  incidents/
```

```

    persons/
    supportTables/
    ...
/domain
    incidents/
    persons/
    ...
repository/
    dto/
    models/
services/
    incidents/
    persons/
    ...

```

- **domain/** - contém os elementos que representam o modelo de dados mapeado para a base de dados relacional.
- **repository/** - contém os modelos de entrada e saída de dados (dto/) e os modelos dos repositórios JPA de cada tabela da base de dados.
- **services/** - contém a implementação da lógica do sistema.
- **api/** - contém os *handlers* de exceções da API (*exceptions/*) e os *controllers* (*login/*, *incidents/*, *persons/*, ...) que expõe todos os *endpoints*, que são abordados na seção "Endpoints".

5.1.1 Entidades

A diretoria `domain/` define o modelo de dados da aplicação. Esta diretoria está também subdividida por áreas funcionais, como `incidents/`, `persons/`, entre outros, que incluem todos os arquivos de cada entidade presente no modelo de dados.

Nas diretorias das subdivisões, encontram-se as entidades JPA que serão mapeadas para entidades da base de dados relacional. Para facilitar este mapeamento, o Jakarta JPA [17] fornece anotações para mapear classes (`@Entity`, `@Table`, etc.), sabendo assim que classe será mapeada para o modelo relacional, e para os mapeamentos de atributos (`@Column`, `@JoinColumn`, etc.), relações (`@OneToMany`, `@ManyToOne`, etc.) e chaves primárias (`@Id`), ilustrado na Figura 5.1.

```

@Entity
@Table(name = "users")
data class User (
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    val userId: Long = 0,
    val userName: String = "",
    val name: String = "",
    val password: String = "",

    @ManyToOne
    @JoinColumn(name = "userProfileId")
    val userProfile: UserProfile = UserProfile(),

    @ManyToOne
    @JoinColumn(name = "userDataProfileId")
    val userDataProfile: UserDataProfile = UserDataProfile(),

    val startDate: LocalDate = LocalDate.now(),
    val endDate: LocalDate? = null,
    val createdAt: LocalDateTime = LocalDateTime.now(),
    val updatedAt: LocalDateTime = LocalDateTime.now(),
)

```

Figura 5.1: Exemplo da entidade relacional dos utilizadores.

5.1.2 DTOs (*Data Transfer Objects*) e Repositórios

Na arquitetura do *backend*, foi adotado o padrão de *Data Transfer Objects* (DTO) para promover uma separação clara entre o modelo de dados interno da aplicação (entidades JPA) e os dados expostos pela API ou recebidos a partir do *frontend*. Um exemplo de um DTO exposto pela API (Figura 5.2a) e um DTO para dados recebidos (Figura 5.2b) estão ilustrados na Figura 5.2.

Os DTOs ajudam a impedir que campos sensíveis (por exemplo, *password* encriptada) ou que dados internos sejam acidentalmente expostos, permitem regras de validação nos dados recebidos, antes de serem convertidos para entidades persistentes e permitem modelos distintos de entrada e saída de dados, adaptados às necessidades específicas de cada *endpoint*.

```

data class UserOutputModel (
    val userId: Long,
    val userName: String,
    val name: String,

    val iserProfile: UserProfile,
    val userDataProfiel: UserDataProfile,

    val startDate: LocalDate,
    val endDate: LocalDate,
    val createdAt: LocalDateTime,
    val updatedAt: LocalDateTime,
)
(a) DTO de saída do utilizador

data class UserInputModel (
    val userName: String,
    val name: String,
    val password: String,
    val startDate: LocalDate,
    val endDate: LocalDate,
)
(b) DTO de entrada do utilizador

```

Figura 5.2: Exemplos dos DTOs dos utilizadores

Quanto ao modelo de repositório, foi usado o JPA em conjunto com *Spring Data JPA* [18] para realizar o mapeamento objeto-relacional (ORM) entre as entidades JPA e as entidades relacionais. As entidades JPA são classes Kotlin que utilizam a anotação Jakarta JPA, referidas na sub-seção 5.1.1, que representam tabelas da base de dados. Na diretoria repository/models/, cada entidade possui uma interface repositório que estende *JpaRepository*, como está ilustrado na Figura 5.3. Este tipo que a interface estende permite aceder a métodos como *findAll()*, *save()*, *deleteById()*, entre outros, sem necessidade de implementar consultas SQL manualmente, reduzindo significativamente o código repetitivo.

Ainda foi discutido o uso da interface *EntityManager* para implementar consultas SQL manualmente, para ter mais controlo sobre as transações, mas a biblioteca do *Spring Data JPA* [18] acabou por ser a escolha óbvia pelo pouco código e muito expressivo, pelo suporte a consultas personalizadas, paginação, pela integração automática com transações e facilitando testes e manutenção.

```

@Repository
@Transactional
interface UserRepository : JpaRepository<User, Long>{
    fun findByUserName(userName: String): List<User>
    @Query(
        """
        SELECT u FROM User u
        WHERE u.startDate <= :targetDate
        AND (u.endDate IS NULL OR u.endDate >= :targetDate)
        """
    )
    fun findValidOnDate(@Param("targetDate") targetDate: LocalDate): List<User>
}

```

Figura 5.3: Exemplo do Repositório dos utilizadores.

5.1.3 Serviços

A diretoria `services/` implementa a lógica da aplicação. Aqui é feita a ponte entre os pedidos à API e a gestão da base de dados. Cada serviço (classe com anotação genérica `@Component` ou `@Service`) comunica com os repositórios para aceder a dados e com os DTOs para tratar os dados de entrada/saída, que são enviados pelos controladores. Aqui é verificado se os dados foram enviados corretamente pelo utilizador ou se as entidades pedidas existem.

5.1.4 Controladores

Os controladores constituem a camada da aplicação responsável por expor os *endpoints* da *Web API REST*, permitindo a comunicação entre os clientes (*frontend*) e a lógica da aplicação de *backend*. Cada controlador está organizado por domínio funcional (*incidents*, *persons*, etc.) refletindo as principais entidades e operações da aplicação de gestão de Zonas de Concentração e Apoio à População (ZCAP). Cada classe é anotada com `@RestController` e `@RequestMapping`, e os métodos usam anotações como `@GetMapping`, `@PostMapping`, `@PutMapping` e `@DeleteMapping` para mapear pedidos HTTP aos métodos correspondentes. Esta abordagem, baseada em convenções *RESTful*, proporciona uma interface clara, estruturada e facilmente utilizável tanto por aplicações cliente como por ferramentas de documentação automática como o *Swagger UI* (secção 5.4).

5.2 Endpoints

A *Web API REST* desenvolvida para esta aplicação define um conjunto de *endpoints* organizados por domínio funcional, permitindo operações de consulta, criação e atualização, sendo a eliminação, apenas para algumas exceções, uma vez que o objetivo é reter todo o histórico, em caso de consulta futura. Estas operações são sobre os principais recursos da aplicação: utilizadores, zonas de concentração e apoio à população (ZCAPs), estrutura hierárquica, pessoas, incidentes e *login/logout*.

As seguintes figuras ilustram a organização dos *endpoints* (Figura 5.4) e um exemplo mais específico para demonstrar um modelo de *endpoints* (Figura 5.5).

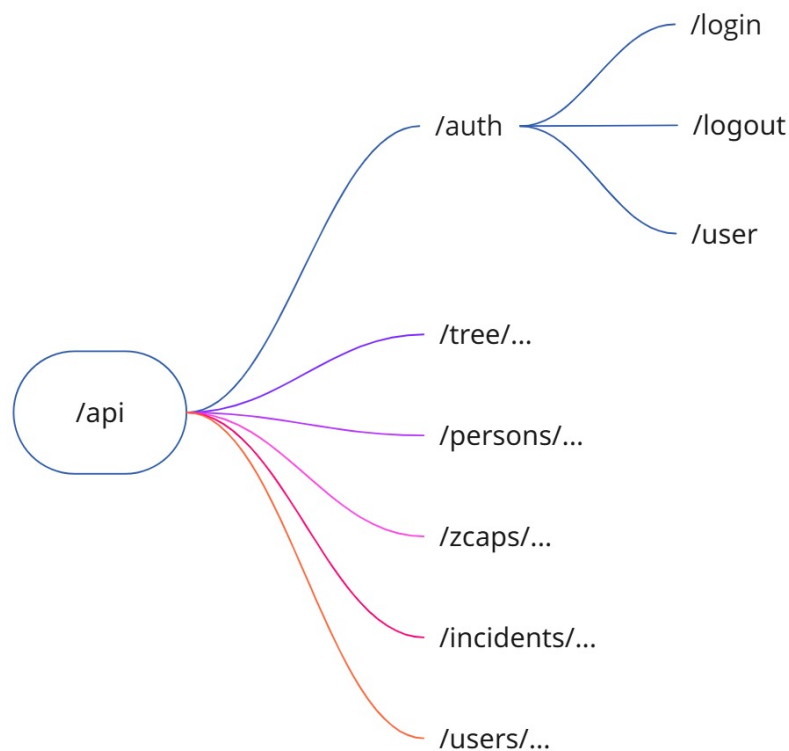


Figura 5.4: Organização dos *Endpoints*.

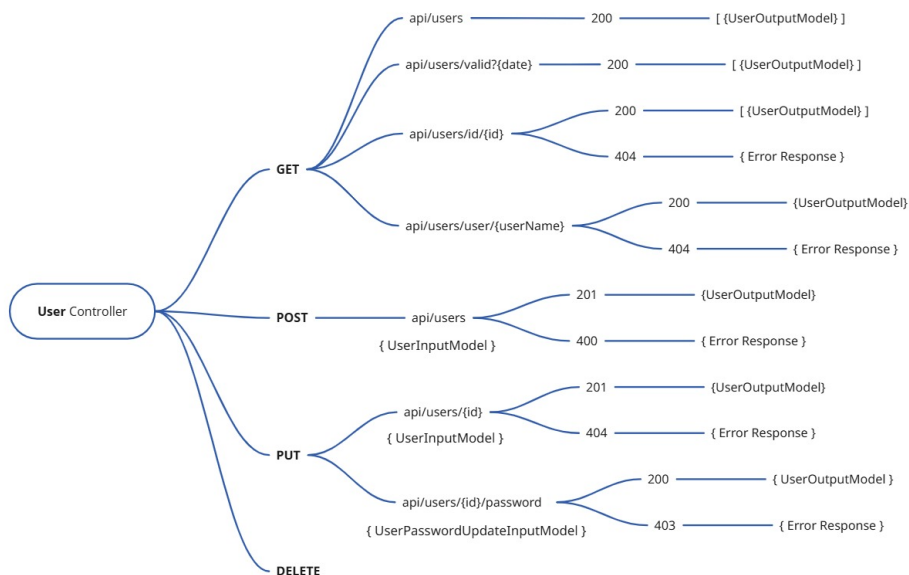


Figura 5.5: Exemplo do controlador dos utilizadores.

Apenas os *endpoints* de `/auth` são públicos. Para os restantes dos *endpoints* é utilizado JWT para autenticação e autorização, uma vez que é *stateless*, ou seja, não requer sessão no servidor, é compacto e transportável, fácil de verificar a autenticidade com chave secreta e

pode conter dados úteis, como informações sobre o utilizador.

Devido ao facto de a aplicação *frontend* (Capítulo 4), por omissão, priorizar os dados *offline-first*, alguns dos *endpoints* apenas serão para uso de clientes externos. Por omissão, foram criados *endpoints* para o retorno de entidades disponíveis numa certa data, entre outros, mas que não teve uso na aplicação, sendo assim estes para uso futuro.

5.3 Estrutura de Respostas

As respostas seguem o formato JSON, com campos bem definidos, mensagens de erro padronizadas e códigos de estado HTTP apropriados (200 OK, 201 Created, 400 *Bad Request*, 401 *Unauthorized*, etc.), facilitando o tratamento no lado do cliente.

5.3.1 Respostas de Sucesso

Em caso de sucesso, apenas será retornado o DTO do elemento pedido ou a lista desses mesmos DTO em formato JSON. Geralmente, tem o seguinte formato:

```
{
  id: 1,
  name: Exemplo1,
  startDate: "2025-01-01",
  endDate: "2027-01-01",
  createdAt: "2025-01-01T12:12:12.12",
  updatedAt: "2025-01-01T12:12:12.12",
}
```

As respostas possíveis em caso de sucesso são:

- 200 *Ok* - quando é retornado o/s elemento/s pedidos.
- 201 *Created* - quando um elemento é armazenado com sucesso, retornando o próprio elemento.
- 204 *No Content* - quando um elemento é eliminado com sucesso.

5.3.2 Respostas de erro

O *Spring Web* [14] fornece anotações como *@ExceptionHandler*, que marca um método que lida com uma exceção específica, e *@RestControllerAdvice*, que define a classe que aplica *@ExceptionHandler* a todos os controladores *REST*. Assim, em caso de erro, é lançada uma exceção tratada pelo respetivo *handler*, retornando um objeto genérico designado por *ErrorMessage* como está no exemplo:

```
{
  errorCode: "ENTITY_NOT_FOUND",
  errorMessage: "The entity with the given ID was not found.",
  details: "Person with ID 99 not found."
}
```

As respostas possíveis em caso de erro são:

- 400 *Bad Request* - quando os dados de entrada estão no formato errado.
- 401 *Unauthorized* - quando o utilizador não tem as credenciais corretas para o uso dos *endpoints*.
- 404 *Not Found* - quando um elemento não é encontrado.
- 409 *Conflict* - quando um elemento já existe na base de dados.
- 500 *Internal Server Error* - quando algum erro é retornado da base de dados.

5.4 Documentação

Para documentação, é utilizado *Swagger* [20], mais especificamente Swagger UI, que fornece uma interface web interativa que permite aos utilizadores visualizar, testar e explorar *Endpoints* de uma API REST de forma intuitiva, diretamente no navegador. Utiliza a especificação OpenAPI [20] como base e gera automaticamente uma documentação dinâmica, onde é possível:

- Visualizar todos os *endpoints* da API com os respetivos métodos (GET, POST, etc.).
- Consultar os parâmetros de entrada, cabeçalhos, corpo da requisição e respostas esperadas.
- Executar chamadas diretamente pela interface (útil para testes e *debug*).
- Ver mensagens de erro, *status codes* e exemplos de *request/response*.

Capítulo 6

Avaliação experimental

A importância da realização de testes no processo de desenvolvimento de software é elevada para garantir a qualidade do sistema desenvolvido. Estes testes visam detetar erros em fases iniciais, validar a implementação de requisitos e assegurar a robustez, funcionalidade e usabilidade de uma aplicação.

Foram incluídos testes unitários quer no *backend* quer no *frontend* embora a sua cobertura seja relativamente baixa comparada com a que se pretendia atingir. Foi realizada uma sessão final de testes com o cliente (representante da ANEPC) que foi encarada como um *User Acceptance Test* (UAT).

6.1 Testes Unitários

6.1.1 Backend

No *backend*, foram realizados testes unitários recorrendo à *framework* de testes do *Spring Boot*, *JUnit*.

Estes testes permitiram garantir o correto comportamento dos componentes de forma isolada e prevenir regressões durante processos de *refactoring* ou adição de novas funcionalidades.

6.1.2 Frontend

No *frontend*, desenvolvido em Flutter, foram utilizados testes unitários e de *widget* com o pacote `flutter_test`.

6.2 Testes de Aceitação com o Cliente

Após a implementação da maioria das funcionalidades, foi agendada e realizada uma sessão de experimentação com o cliente. Esta sessão, vista como um **Teste de Aceitação do Utilizador (User Acceptance Test - UAT)** focou apenas a componente de *frontend* não estando disponível qualquer infraestrutura disponível para testar o *backend* ou a sincronização de dados.

Durante esta fase, o cliente teve oportunidade de interagir diretamente com a aplicação, explorando as diferentes funcionalidades. O objetivo foi validar se o sistema correspondia aos requisitos funcionais e não funcionais inicialmente definidos.

No decorrer da sessão, foram registadas as seguintes observações:

- O cliente confirmou que a aplicação satisfaz os requisitos funcionais principais.
- Foram identificadas pequenas melhorias relacionadas com a usabilidade e *feedback* visual.
- Algumas sugestões registadas e já integradas:
 - Alterar formulário de admissão de pessoas (remover campos desnecessários na primeira fase de admissão).
 - Corrigir possibilidade de selecionar uma data fim anterior à data início.
 - Trocar a ordem do campo descrição nos ecrãs de "Necessidades especiais" e "Apoio necessário".
 - Adicionar a possibilidade de relacionar Pessoal pelo grau de parentesco.
 - Resumo das pessoas acolhidas.
 - Apresentar informação no cartão, se o utente está ou não em ZCAP.
- Algumas sugestões foram identificadas e adicionadas como Trabalho Futuro:
 - Adicionar janela de confirmação ao encerrar a aplicação (ALT+F4 ou botão Fechar na janela do sistema operativo).
 - Possibilidade de importar/exportar dados para ficheiro.
 - Adicionar no menu principal *dashboard* com informações gerais, relevantes no momento da utilização da aplicação.
 - Adicionar no menu principal um botão de atalho para criação de um novo utilizador (com permissões iguais ou inferiores ao utilizador que cria).
 - Adicionar no menu principal um botão de atalho que permita criar uma nova ZCAP.
 - Adicionar no menu principal um botão de atalho que permita iniciar um novo incidente.
 - Gravar o utilizador que faz o registo, ou criar campo do técnico que faz o registo no acolhimento.

A realização deste teste permitiu não só validar o trabalho realizado, mas também alinhar as expectativas entre o cliente e o produto final entregue.

Capítulo 7

Conclusão

Ao longo do projeto, foi desenvolvida uma aplicação para gestão de Zonas de Concentração e Apoio à População (ZCAP) em cenários de emergência, com o objetivo de facilitar a coordenação e o apoio a populações deslocadas.

A solução é composta por um *backend* desenvolvido em Kotlin [16] com *Spring Framework* [13], que expõe uma *Web API RESTful*, e um *frontend* desenvolvido com *Flutter* [5], compatível com dispositivos móveis e ambientes *desktop*.

A persistência local foi assegurada através da utilização da base de dados Isar [8], enquanto o *backend* utiliza um modelo relacional com JPA [17]. Foram ainda integrados mecanismos de autenticação baseados em JWT [19] e uma documentação interativa da API via Swagger/OpenAPI [20].

Foi possível criar uma solução funcional e adaptável para a gestão de Zonas de Concentração e Apoio à População (ZCAP), com foco na operação em contextos de emergência. A arquitetura modular tem a separação clara entre o *frontend* e o *backend*, facilitando o desenvolvimento e os testes de forma independente.

Foram ultrapassados diversos desafios técnicos, nomeadamente a sincronização de dados entre o *frontend* e o *backend* em cenários de conectividade intermitente, a gestão da persistência local com Isar e a configuração segura do sistema de autenticação baseada em JWT.

A integração da documentação interativa via Swagger/OpenAPI revelou-se útil tanto para testes como para futuras manutenções ou integrações externas. Globalmente, o projeto contribui com uma base sólida para sistemas de apoio à população em situações críticas, com potencial para evoluir e ser adaptado a diferentes contextos operacionais.

Os testes realizados com pessoas da área do domínio da aplicação mostraram a adequação da solução proposta, indicando também pontos de expansão e melhoria.

Foram atingidos todos os objetivos definidos na proposta de projeto, à exceção da criação de *endpoints* específicos com dados estatísticos para fins de relatório, os *endpoints* disponibilizados retornam toda a informação relativa ao pedido ou apenas a que é válida na data fornecida (*endpoints* ../valid). O sistema desenvolvido cumpre integralmente os requisitos funcionais e técnicos estabelecidos, demonstrando a viabilidade da solução proposta, bem

como a sua aplicabilidade prática.

Embora os objetivos tenham sido, maioritariamente, alcançados, existem oportunidades de melhoria contínua do produto que permitem complementar e melhorar a solução implementada. Em particular, destaca-se a possibilidade de concluir a funcionalidade não implementada nesta fase, nomeadamente, a criação de *endpoints* com dados estatísticos para fins de relatório. Para além disso, será relevante:

- Desenvolver um interface de utilizador para resolução de conflitos caso existam. A estratégia implementada utiliza as datas da última atualização do registo para decidir qual dos registos manter mas esta estratégia não é a mais eficiente pois podem existir alterações simultâneas em ambos os lados e existirá perda de dados.
- Possibilidade de promover um dos computadores da ZCAP assumir temporariamente o papel de *host local*. Este computador funcionaria como ponto de coordenação e partilha de dados entre os restantes dispositivos na rede local (por exemplo, via *Wi-Fi Direct* ou rede local ad-hoc), assegurando a continuidade das operações colaborativas em modo *offline*.
- A funcionalidade de filtragem de **Acesso a dados** (ver secção 4.3.7) foi desenvolvida apenas no *frontend* mas deve ser suportada de igual forma pelo *backend* para permitir a partilha dos *endpoints* da Web API para clientes externos de forma controlada.
- Melhorar o serviço de sincronização, implementando um mecanismo de retentativa em caso de falha na comunicação com a Web API, recorrendo a um algoritmo de *exponential backoff* por forma a gerir com mais eficiência os tempos de espera entre tentativas consecutivas e reduzir a sobrecarga do sistema.
- Melhorar a quantidade de informação armazenada, mantendo localmente apenas a informação necessária para o seu normal funcionamento (suprimir localmente dados de histórico).
- Sugestões reportadas pelo cliente:
 - Possibilidade de importar\exportar dados para ficheiro. Esta necessidade nasce do facto de se ter concluído que dar visibilidade\acesso ao *backend* dentro da rede protegida da ANEPC é um processo complexo e moroso pelo que uma alternativa de sincronização via import\export manual de ficheiros viabiliza a utilização imediata do produto.
 - Adicionar janela de confirmação ao encerrar a aplicação (ALT+F4 ou botão Fechar na janela do sistema operativo).
 - Adicionar no menu principal *dashboard* com informações gerais, relevantes no momento da utilização da aplicação.

- Adicionar no menu principal um botão de atalho para criação de um novo utilizador (com permissões iguais ou inferiores ao user que cria).
- Adicionar no menu principal um botão de atalho que permita criar uma nova ZCAP.
- Adicionar no menu principal um botão de atalho que permita iniciar um novo incidente.
- Gravar o utilizador que faz o registo, ou criar campo do técnico que faz o registo no acolhimento.

Referências

- [1] Presidência do Conselho de Ministros. Decreto-lei n.º 90-a/2022. <https://diariodarepublica.pt/dr/detalhe/decreto-lei/90-a-2022-205450974>, 2022. Publicação: Diário da República n.º 251/2022, 1º Suplemento, Série I de 2022-12-30, páginas 8-22. último acesso em julho de 2025.
- [2] Presidência do Conselho de Ministros. Despacho n.º 4067/2024. <https://diariodarepublica.pt/dr/detalhe/despacho/4067-2024-860479489>, 2024. Publicação: Diário da República n.º 74/2024, Série II de 2024-04-15. último acesso em julho de 2025.
- [3] Autoridade Nacional de Emergência e Proteção Civil (ANEPC). Caderno técnico prociv 32 – instalação e gestão de zonas de concentração e apoio à população (ZCAP). https://prociv.gov.pt/media/uqtisk2t/ct_prociv32_zcap.pdf, 2025. Acesso em julho de 2025.
- [4] Autoridade Nacional de Emergência e Proteção Civil (ANEPC). Sigo - sistema integrado de gestão de operações. <https://app.sigo-online.pt>, 2025. Acesso em julho de 2025.
- [5] Google. Flutter – build apps for any screen. <https://flutter.dev>, 2025. Acesso em: julho 2025.
- [6] Mark Gamble. Offline-first: uma mentalidade para desenvolver aplicativos móveis mais rápidos e confiáveis. <https://www.couchbase.com/blog/pt/offline-first-more-reliable-mobile-apps/>, 2023. Acesso em: julho de 2025.
- [7] YLD. Why your application needs to be offline-first. <https://www.yld.io/blog/why-your-app-should-be-offline-first-use-cases>, 2016. Acesso em: julho de 2025.
- [8] Simon Choi. Isar Database. <https://isar.dev/>, 2023. Acesso em: julho 2025.
- [9] Peter Chen. The entity-relationship model—toward a unified view of data. <https://dl.acm.org/doi/pdf/10.1145/320434.320440>, 1990. Acesso em: julho 2025.
- [10] Microsoft Corporation. Microsoft SQL Server. <https://www.microsoft.com/sql-server>, 2025. Acesso em: julho 2025.

- [11] Microsoft Corporation. SQL Server Management Studio (SSMS). <https://learn.microsoft.com/sql/ssms>, 2025. Acesso em: julho 2025.
- [12] CubiVue. animated_tree_view 2.3.0. https://pub.dev/packages/animated_tree_view, 2024. Acesso em: julho de 2025.
- [13] Spring Team. Spring framework documentation. <https://docs.spring.io/spring-framework/reference/index.html>, 2025. Acesso em: julho de 2025.
- [14] Spring Team. Spring framework documentation. <https://docs.spring.io/spring-framework/reference/web/webmvc.html>, 2025. Acesso em: julho de 2025.
- [15] Spring Team. Spring boot reference documentation. <https://docs.spring.io/spring-boot/docs/current/reference/html/>, 2025. Acesso em: julho de 2025.
- [16] JetBrains. Kotlin language documentation. <https://kotlinlang.org/docs/home.html>, 2025. Acesso em: julho de 2025.
- [17] Jakarta EE. Jakarta persistence api specification. <https://jakarta.ee/specifications/persistence/>, 2025. Acesso em: julho de 2025.
- [18] Spring Team. Spring data jpa documentation. <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>, 2025. Acesso em: julho de 2025.
- [19] Baeldung. Guide to jwt with spring security. <https://jwt.io/>, 2025. Acesso em: julho de 2025.
- [20] OpenAPI Initiative. Openapi specification. <https://swagger.io/specification/>, 2025. Acesso em: julho de 2025.

Apêndice A

Lista de *Endpoints* da Web API

Este apêndice apresenta os *endpoints* desenvolvidos para a API REST do sistema, incluindo o método HTTP, o caminho e uma breve descrição da sua funcionalidade. Estes *endpoints* permitem a comunicação entre o *frontend* e o *backend* da aplicação ZCAPNet podendo ser também utilizados por clientes externos quando devidamente autenticados.

A.1 Incidentes

Nesta secção estão representados todos os *endpoints* relacionados com os incidentes, assim como o tipo de incidentes (tabela A.1), os incidentes (tabela A.2), a lista de ZCAPs de um incidente (tabela A.3) e a lista de pessoas em cada incidente-ZCAP (tabela A.4).

Método	Endpoint	Descrição
GET	/api/incident-types	Obtém a lista de todos os tipos de incidentes.
GET	/api/incident-types/:id	Obtém um tipo de incidente pelo id.
POST	/api/incident-types	Cria um novo tipo de incidente.
PUT	/api/incident-types/:id	Atualiza um tipo de incidente pelo id.
GET	/api/incidents-types/valid	Obtém todos os tipos de incidentes válidos numa data fornecida.

Tabela A.1: Tabela de *endpoints* relativos aos tipos de incidentes.

Método	Endpoint	Descrição
GET	/api/incidents	Obtém a lista de todos as incidentes.
GET	/api/incidents/:id	Obtém um incidente pelo id.
POST	/api/incidents	Cria um novo incidente.
PUT	/api/incidents/:id	Atualiza um incidente pelo id.
GET	/api/incidents/valid	Obtém todos os incidentes válidos numa data fornecida.
GET	/api/incidents/incident-types/:id	Obtém todos os incidentes pelo tipo de incidente fornecido.

Tabela A.2: Tabela de *endpoints* relativos aos incidentes.

Método	Endpoint	Descrição
GET	/api/incident-zcaps	Lista as ZCAPs associadas a um incidente.
GET	/api/incident-zcaps/:id	Obtém uma ZCAP associada a um incidente pelo id.
POST	/api/incident-zcaps	Regista a ativação de uma ZCAP para um incidente.
PUT	/api/incident-zcaps/:id	Atualiza uma ZCAP associada a um incidente existente.
GET	/api/incident-zcaps/valid	Lista as ZCAPs associadas a um incidente válidas numa data fornecida.
GET	/api/incident-zcaps/entities/:id	Lista as ZCAPs associadas a um incidente com a entidade fornecida.
GET	/api/incident-zcaps/zcaps/:id	Lista as ZCAPs associadas a um incidente com a ZCAP fornecida.
GET	/api/incident-zcaps/incidents/:id	Lista as ZCAPs associadas a um incidente com o incidente fornecido.

Tabela A.3: Tabela de *endpoints* relativos às ZCAPs associadas a um incidente.

Método	Endpoint	Descrição
GET	/api/incident-zcap-persons	Lista as pessoas associadas a um incidente-ZCAP.
GET	/api/incident-zcap-persons/:id	Obtém uma pessoa associada a um incidente-ZCAP pelo id.
POST	/api/incident-zcap-persons	Regista a entrada de uma pessoa num incidente-ZCAP.
PUT	/api/incident-zcap-persons/:id	Atualiza uma pessoa associada a um incidente-ZCAP existente.
GET	/api/incident-zcap-persons/valid	Lista as pessoas associadas a um incidente-ZCAP válidas numa data fornecida.
GET	/api/incident-zcap-persons/incident-zcaps/:id	Lista pessoas associadas a um incidente-ZCAP pelo incidente-ZCAP fornecido.
GET	/api/incident-zcap-persons/persons/:id	Lista as pessoas associadas a um incidente-ZCAP pela pessoa fornecida.

Tabela A.4: Tabela de *endpoints* relativos às pessoas associadas a um incidente-ZCAP.

A.2 Pessoas

Nesta secção estão representados todos os *endpoints* relacionados com as pessoas, assim como os destinos de partida das ZCAPs (tabela A.5), as pessoas (tabela A.6), as necessidades especiais (tabela A.7) e a lista de necessidades especiais por pessoa (tabela A.8), os suportes necessários (tabela A.9) e a lista de suportes necessários por pessoa (tabela A.10), os tipos de grau de parentesco (tabela A.11) e os graus de parentesco entre pessoas (tabela A.12).

Método	Endpoint	Descrição
GET	/api/departure-destinations	Obtém a lista de todos os destinos.
GET	/api/departure-destinations/:id	Obtém um destino pelo id.
POST	/api/departure-destinations	Cria um novo destino.
PUT	/api/departure-destinations/:id	Atualiza um destino pelo id.
DELETE	/api/departure-destinations/:id	Elimina um destino existente
GET	/api/departure-destinations/valid	Obtém todos os destinos válidos numa data fornecida.

Tabela A.5: Tabela de *endpoints* relativos destinos de partida de uma ZCAP.

Método	Endpoint	Descrição
GET	/api/persons	Obtém a lista de todas as pessoas.
GET	/api/persons/:id	Obtém uma pessoa pelo id.
POST	/api/persons	Cria uma nova pessoa.
PUT	/api/persons/:id	Atualiza uma pessoa pelo id.
DELETE	/api/persons/:id	Elimina uma pessoa existente.
GET	/api/persons/name	Obtém todas as pessoas que contêm o nome fornecido.

Tabela A.6: Tabela de *endpoints* relativos às pessoas.

Método	Endpoint	Descrição
GET	/api/special-needs	Obtém a lista de todas as necessidades especiais.
GET	/api/special-needs/:id	Obtém uma necessidade especial pelo id.
POST	/api/special-needs	Cria uma nova necessidade especial.
PUT	/api/special-needs/:id	Atualiza uma necessidade especial pelo id.
DELETE	/api/special-needs/:id	Elimina uma necessidade especial existente.
GET	/api/special-needs/valid	Obtém todas as necessidades especiais válidos na data fornecida.

Tabela A.7: Tabela de *endpoints* relativos às necessidades especiais.

Método	Endpoint	Descrição
GET	/api/person-special-needs	Obtém a lista de todas as necessidades especiais de todas as pessoas.
GET	/api/person-special-needs/:id	Obtém uma necessidade especial de uma pessoa pelo id.
POST	/api/person-special-needs	Cria uma nova necessidade especial para uma pessoa.
PUT	/api/person-special-needs/:id	Atualiza uma necessidade especial de uma pessoa pelo id.
DELETE	/api/person-special-needs/:id	Elimina uma necessidade especial de uma pessoa existente.
GET	/api/person-special-needs/valid	Obtém todas as necessidades especiais de cada pessoa válidas na data fornecida.
GET	/api/person-special-needs/person/:id	Obtém todas as necessidades especiais da pessoa fornecida.

Tabela A.8: Tabela de *endpoints* relativos à lista de necessidades especiais de cada pessoa.

Método	Endpoint	Descrição
GET	/api/support-needed	Obtém a lista de todos os suportes necessários.
GET	/api/support-needed/:id	Obtém um suporte necessário pelo id.
POST	/api/support-needed	Cria um novo suporte necessário.
PUT	/api/support-needed/:id	Atualiza um suporte necessário pelo id.
DELETE	/api/support-needed/:id	Elimina um suporte necessário existente.
GET	/api/support-needed/valid	Obtém todos os suportes necessários válidos na data fornecida.

Tabela A.9: Tabela de *endpoints* relativos aos suportes necessários.

Método	Endpoint	Descrição
GET	/api/person-support-needed	Obtém a lista de todos os suportes necessários de todas as pessoas.
GET	/api/person-support-needed/:id	Obtém um suporte necessário de uma pessoa pelo id.
POST	/api/person-support-needed	Cria um novo suporte necessário para uma pessoa.
PUT	/api/person-support-needed/:id	Atualiza um suporte necessário de uma pessoa pelo id.
DELETE	/api/person-support-needed/:id	Elimina um suporte necessário de uma pessoa existente.
GET	/api/person-support-needed/valid	Obtém todos os suportes necessários de cada pessoa válidas na data fornecida.
GET	/api/person-support-needed/person/:id	Obtém todos os suportes necessários da pessoa fornecida.

Tabela A.10: Tabela de *endpoints* relativos à lista de suporte necessário de cada pessoa.

Método	Endpoint	Descrição
GET	/api/relation-type	Obtém a lista de todos os tipos de graus de parentesco.
GET	/api/relation-type/:id	Obtém um tipo de grau de parentesco pelo id.
POST	/api/relation-type	Cria um novo tipo de grau de parentesco.
PUT	/api/relation-type/:id	Atualiza um tipo de grau de parentesco pelo id.
DELETE	/api/relation-type/:id	Elimina um tipo de grau de parentesco existente.
GET	/api/relation-type/valid	Obtém todos os tipos de graus de parentesco válidos na data fornecida.

Tabela A.11: Tabela de *endpoints* relativos ao tipos de graus de parentesco.

Método	Endpoint	Descrição
GET	/api/relations	Obtém a lista de todos os de graus de parentesco existentes.
GET	/api/relations/:id	Obtém um grau de parentesco pelo id.
POST	/api/relations	Cria um novo grau de parentesco entre pessoas.
PUT	/api/relations/:id	Atualiza um grau de parentesco pelo id.
DELETE	/api/relations/:id	Elimina um grau de parentesco existente.
GET	/api/relations/person1/:id	Obtém todos os graus de parentesco da pessoas fornecida.

Tabela A.12: Tabela de *endpoints* relativos aos graus de parentesco entre pessoas.

A.3 ZCAPs

Nesta secção estão representados todos os *endpoints* relacionados com as ZCAPs, assim como os tipos de edifício das ZCAPs (tabela A.13), os tipos de entidades (tabela A.14), as entidades que gerem as ZCAPs (tabela A.15), as ZCAPs (tabela A.16), as categorias dos detalhes das ZCAPs (tabela A.17), os tipos de detalhes das ZCAPs (tabela A.18) e os detalhes das ZCAPs (tabela A.19).

Método	Endpoint	Descrição
GET	/api/buildingTypes	Obtém a lista de todos os tipos de edifício existentes.
GET	/api/buildingTypes/:id	Obtém um tipo de edifício pelo id.
POST	/api/buildingTypes	Cria um novo tipo de edifício.
PUT	/api/buildingTypes/:id	Atualiza um tipo de edifício pelo id.
GET	/api/buildingTypes/valid	Obtém todos os tipos de edifício válidos na data fornecida.

Tabela A.13: Tabela de *endpoints* relativos aos tipos de edifício de ZCAPs.

Método	Endpoint	Descrição
GET	/api/entityTypes	Obtém a lista de todos os tipos de entidade existentes.
GET	/api/entityTypes/:id	Obtém um tipo de entidade pelo id.
POST	/api/entityTypes	Cria um novo tipo de entidade.
PUT	/api/entityTypes/:id	Atualiza um tipo de entidade pelo id.
GET	/api/entityTypes/valid	Obtém todos os tipos de entidade válidas na data fornecida.

Tabela A.14: Tabela de *endpoints* relativos aos tipos de entidade que gerem uma ZCAP.

Método	Endpoint	Descrição
GET	/api/entities	Obtém a lista de todas as entidades existentes.
GET	/api/entities/:id	Obtém uma entidade pelo id.
POST	/api/entities	Cria uma nova entidade.
PUT	/api/entities/:id	Atualiza uma entidade pelo id.
GET	/api/entities/valid	Obtém todas as entidade válidas na data fornecida.

Tabela A.15: Tabela de *endpoints* relativos às entidades que gerem uma ZCAP.

Método	Endpoint	Descrição
GET	/api/zcaps	Obtém a lista de todas as ZCAPs existentes.
GET	/api/zcaps/:id	Obtém uma ZCAP pelo id.
POST	/api/zcaps	Cria uma nova ZCAP.
PUT	/api/zcaps/:id	Atualiza uma ZCAP pelo id.
GET	/api/zcaps/valid	Obtém todas as ZCAPs válidas na data fornecida.
GET	/api/zcaps/building-types/:id	Obtém todas as ZCAPs pelo tipo de edifício fornecido.
GET	/api/zcaps/entities/:id	Obtém todas as ZCAPs pela entidade fornecida.

Tabela A.16: Tabela de *endpoints* relativos às ZCAPs.

Método	Endpoint	Descrição
GET	/api/detail-type-categories	Obtém a lista de todas as categorias existentes.
GET	/api/detail-type-categories/:id	Obtém uma categoria pelo id.
POST	/api/detail-type-categories	Cria uma nova categoria.
PUT	/api/detail-type-categories/:id	Atualiza uma categoria pelo id.
GET	/api/detail-type-categories/valid	Obtém todas as categorias válidas na data fornecida.

Tabela A.17: Tabela de *endpoints* relativos às categorias dos detalhes das ZCAPs.

Método	Endpoint	Descrição
GET	/api/zcap-detail-types	Obtém a lista de todos os tipos de detalhe existentes.
GET	/api/zcap-detail-types/:id	Obtém um tipo de detalhe pelo id.
POST	/api/zcap-detail-types	Cria um novo tipo de detalhe.
PUT	/api/zcap-detail-types/:id	Atualiza um tipo de detalhe pelo id.
GET	/api/zcap-detail-types/valid	Obtém todos os tipos de detalhe válidos na data fornecida.

Tabela A.18: Tabela de *endpoints* relativos aos tipos de detalhe das ZCAPs.

Método	Endpoint	Descrição
GET	/api/zcap-details	Obtém a lista de todos os detalhes existentes.
GET	/api/zcap-details/:id	Obtém um detalhe pelo id.
POST	/api/zcap-details	Cria um novo detalhe.
PUT	/api/zcap-details/:id	Atualiza um detalhe pelo id.
GET	/api/zcap-details/valid	Obtém todos os detalhes válidos na data fornecida.

Tabela A.19: Tabela de *endpoints* relativos aos detalhes das ZCAPs.

A.4 Árvore hierárquica

Nesta secção estão representados todos os *endpoints* relacionados com a árvore hierárquica, assim como os níveis das divisões administrativas (tabela A.20), os elementos da árvore (tabela A.21), os tipos de detalhes dos elementos da árvore (tabela A.24), os detalhes dos elementos da árvore (tabela A.23) e a associação do nível com o tipo de detalhe, para que seja possível definir detalhes apenas para um nível administrativo (tabela A.24).

Método	Endpoint	Descrição
GET	/api/tree-levels	Obtém a lista de todos os níveis existentes.
GET	/api/tree-levels/:id	Obtém um nível pelo id.
POST	/api/tree-levels	Cria um novo nível.
PUT	/api/tree-levels/:id	Atualiza um nível pelo id.
DELETE	/api/tree-levels/:id	Elimina um nível existente.
GET	/api/tree-levels/valid	Obtém todos os níveis válidos na data fornecida.

Tabela A.20: Tabela de *endpoints* relativos aos níveis das divisões administrativas.

Método	Endpoint	Descrição
GET	/api/trees	Obtém a lista de todos os elementos existentes.
GET	/api/trees/:id	Obtém um elemento pelo id.
POST	/api/trees	Cria um novo elemento.
PUT	/api/trees/:id	Atualiza um elemento pelo id.
DELETE	/api/trees/:id	Elimina um elemento existente.
GET	/api/trees/valid	Obtém todos os elementos válidos na data fornecida.
GET	/api/trees/treeLevel/:id	Obtém todos os elementos pelo nível fornecido.
GET	/api/trees/children/:id	Obtém todos os filhos diretos do elemento fornecido.
GET	/api/trees/allChildren/:id	Obtém todos os filhos do elemento fornecido.

Tabela A.21: Tabela de *endpoints* relativos aos elementos da árvore hierárquica.

Método	Endpoint	Descrição
GET	/api/tree-record-detail-types	Obtém a lista de todos os tipos de detalhe existentes.
GET	/api/tree-record-detail-types/:id	Obtém um tipo de detalhe pelo id.
POST	/api/tree-record-detail-types	Cria um tipo de detalhe.
PUT	/api/tree-record-detail-types/:id	Atualiza um tipo de detalhe pelo id.
DELETE	/api/tree-record-detail-types/:id	Elimina um tipo de detalhe existente.
GET	/api/tree-record-detail-types/valid	Obtém todos os tipos de detalhe válidos na data fornecida.

Tabela A.22: Tabela de *endpoints* relativos aos tipos de detalhe dos elementos da árvore hierárquica.

Método	Endpoint	Descrição
GET	/api/tree-record-details	Obtém a lista de todos os detalhes existentes.
GET	/api/tree-record-details/:id	Obtém um detalhe pelo id.
POST	/api/tree-record-details	Cria um detalhe.
PUT	/api/tree-record-details/:id	Atualiza um detalhe pelo id.
DELETE	/api/tree-record-details/:id	Elimina um detalhe existente.
GET	/api/tree-record-details/valid	Obtém todos os detalhes válidos na data fornecida.
GET	/api/tree-record-details/detailType/:id	Obtém todos os detalhes pelo tipo fornecido.

Tabela A.23: Tabela de *endpoints* relativos aos detalhes dos elementos da árvore hierárquica.

Método	Endpoint	Descrição
GET	/api/tree-level-detail-type	Obtém a lista de todas as associações existentes.
GET	/api/tree-level-detail-type/:id	Obtém uma associação pelo id.
POST	/api/tree-level-detail-type	Cria uma associação.
PUT	/api/tree-level-detail-type/:id	Atualiza uma associação pelo id.
DELETE	/api/tree-level-detail-type/:id	Elimina uma associação existente.
GET	/api/tree-level-detail-type/valid	Obtém todas as associações válidas na data fornecida.
GET	/api/tree-level-detail-type/treeLevel/:id	Obtém todas as associações pelo nível fornecido.
GET	/api/tree-level-detail-type/detailType/:id	Obtém todas as associações pelo tipo de detalhe fornecido.

Tabela A.24: Tabela de *endpoints* relativos à associação do nível como o tipo de detalhes dos elementos da árvore hierárquica.

A.5 Utilizadores

Nesta secção estão representados todos os *endpoints* relacionados com os utilizadores, assim como os perfis de dados (tabela A.25) e seus detalhes (tabela A.26), para que cada perfil seja associado a uma zona administrativa, os perfis de utilizadores (tabela A.27), as chaves de acesso aos menus (tabela A.28), as permissões de cada perfil (tabela A.29) e os utilizadores para uso de clientes exteriores (tabela A.30) e para uso da aplicação *offline-first* (tabela A.31).

Método	Endpoint	Descrição
GET	/api/users/dataprofiles	Obtém a lista de todos os perfis de dados existentes.
GET	/api/users/dataprofiles/:id	Obtém um perfil de dados pelo id.
POST	/api/users/dataprofiles	Cria um perfil de dados.
PUT	/api/users/dataprofiles/:id	Atualiza um perfil de dados pelo id.
GET	/api/users/dataprofiles/valid	Obtém todos os perfis de dados válidos na data fornecida.

Tabela A.25: Tabela de *endpoints* relativos aos perfis de dados dos utilizadores.

Método	Endpoint	Descrição
GET	/api/users/dataprofiles/details	Obtém a lista de todos os detalhes de perfis de dados existentes.
GET	/api/users/dataprofiles/:id/details	Obtém um detalhe de perfil de dados pelo id.
POST	/api/users/dataprofiles/detail	Cria um detalhe de perfil de dados.
DELETE	/api/users/dataprofiles/:id/detail/:treeId	Elimina um detalhe de perfil de dados pelo id do perfil e do elemento da árvore.

Tabela A.26: Tabela de *endpoints* relativos aos detalhes dos perfis de dados dos utilizadores.

Método	Endpoint	Descrição
GET	/api/users/profiles	Obtém a lista de todos os perfis existentes.
GET	/api/users/profiles/:id	Obtém um perfil pelo id.
POST	/api/users/profiles	Cria um perfil.
PUT	/api/users/profiles/:id	Atualiza um perfil pelo id.
GET	/api/users/profiles/valid	Obtém a lista de perfis válidos na data fornecida.

Tabela A.27: Tabela de *endpoints* relativos aos perfis dos utilizadores.

Método	Endpoint	Descrição
GET	/api/users/access-keys	Obtém a lista de todas as chaves existentes.
GET	/api/users/access-keys/:id	Obtém uma chave pelo id.
POST	/api/users/access-keys	Cria uma chave.
PUT	/api/users/access-keys/:id	Atualiza uma chave pelo id.

Tabela A.28: Tabela de *endpoints* relativos às chaves de acesso aos menus.

Método	Endpoint	Descrição
GET	/api/users/profiles/allowances	Obtém a lista de todas as permissões existentes.
GET	/api/users/profiles/allowances/:id	Obtém uma permissão pelo id.

Tabela A.29: Tabela de *endpoints* relativos à lista de permissões de cada perfil de utilizador.

Método	Endpoint	Descrição
GET	/api/users	Obtém a lista de todos os utilizadores existentes.
GET	/api/users/:id	Obtém um utilizador pelo id.
POST	/api/users	Cria um utilizador.
PUT	/api/users/:id	Atualiza um utilizador pelo id.
GET	/api/users/:userName	Obtém utilizadores pelo nome fornecido.
GET	/api/users/valid	Obtém os utilizadores válidos na data fornecida.
PUT	/api/users/:id/password	Atualiza a <i>password</i> do utilizador fornecido.

Tabela A.30: Tabela de *endpoints* relativos aos utilizadores.

Método	Endpoint	Descrição
GET	/api/syncable/users	Obtém a lista de todos os utilizadores existentes.
GET	/api/syncable/users/:id	Obtém um utilizador pelo id.
POST	/api/syncable/users	Cria um utilizador.
PUT	/api/syncable/users/:id	Atualiza um utilizador pelo id.
GET	/api/syncable/users/:userName	Obtém utilizadores pelo nome fornecido.
GET	/api/syncable/users/valid	Obtém os utilizadores válidos na data fornecida.
PUT	/api/syncable/users/:id/password	Atualiza a <i>password</i> do utilizador fornecido.

Tabela A.31: Tabela de *endpoints* relativos aos utilizadores para uso da aplicações *offline-first*.

A.6 Outros endpoints

Nesta secção está representado o resto dos *endpoints* na tabela A.32. Aqui encontram-se os *endpoints* públicos, nomeadamente *login*, *logout*, informações sobre o próprio utilizador e *ping*, para verificação do estado da Web API.

Método	Endpoint	Descrição
POST	/api/auth/login	Tentativa de <i>login</i> de um utilizador.
POST	/api/auth/logout	<i>Logout</i> de um utilizador.
GET	/api/auth/user	Obtém informações sobre o utilizador.
GET	/api/ping	Verificação do estado da Web API.

Tabela A.32: Tabela de *endpoints* relativos aos dados do utilizador.